



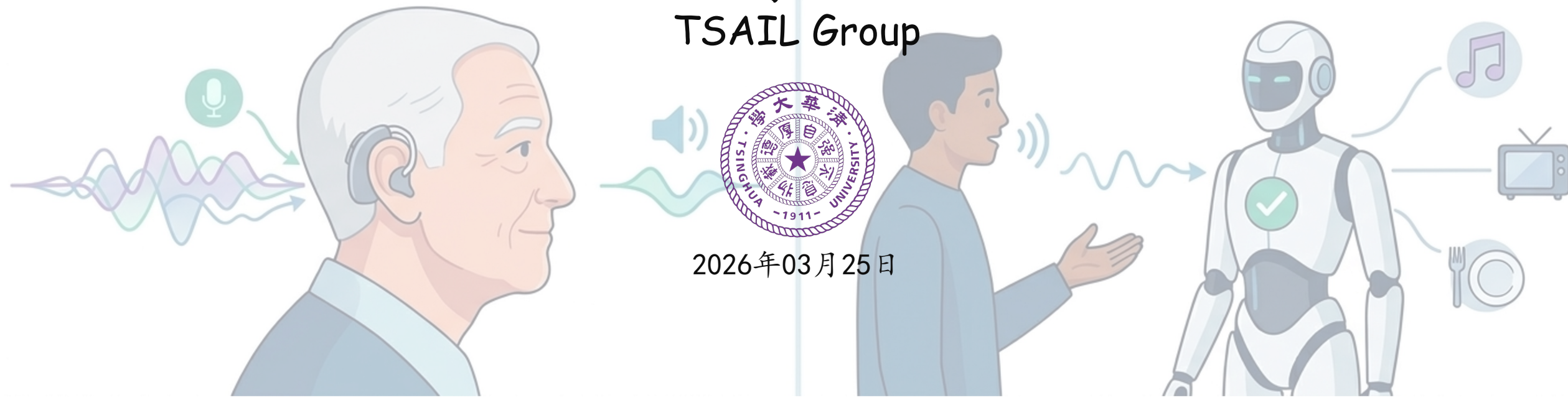
高效语音分离： 架构、融合、因果性与数据 ——从高性能模型到可部署分离系统

李凯

TSAIL Group



2026年03月25日





为什么现在要讲“高效语音分离”

为什么重要

- ✓ ASR的关键前处理，可提升嘈杂环境下识别效果。
- ✓ 适用于会议转写、在线会议、多说话人场景。
- ✓ 也是语音交互、可穿戴和流式应用的重要基础能力。

语音分离正在从
“benchmark 任务”走向
“系统模块”，效率
因此成为核心问题



为什么必须要高效

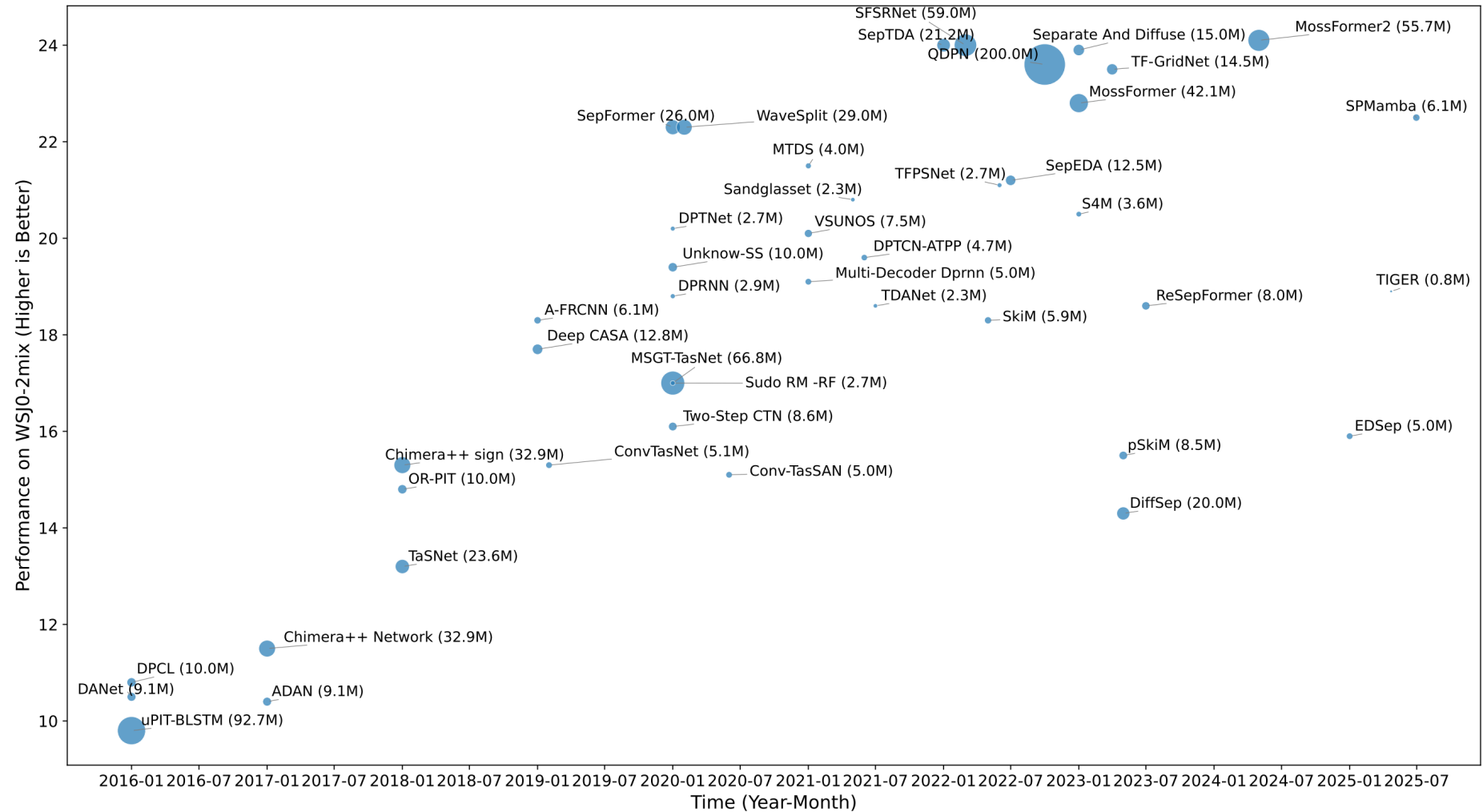
- ✓ 现有研究多关注实验室上的分离质量，部署需求常被忽视。
- ✓ 真实系统要求低时延、低算力、低内存、实时运行。
- ✓ benchmark与真实场景仍存在差距，部署更具挑战。





从“分得开”到“跑得动”

- ✓ 传统文献多强调SI-SDR/SDR。
- ✓ 但真实应用还要求：
低参数、低 MACs、
低时延、强泛化。





语音分离中的效率到底来自哪里

结构 (轻量骨干/模块替换/剪枝压缩)

- 异步更新
- Top-down Attention
- SSM替代Transformer
- 结构化剪枝压缩高层开销

融合 (信息流设计/融合位置/融合算子)

- 早期融合优于后期融合
 - 低层特征融合更有效
- 复杂门控可替换为求和融合
- 重排跨模态信息进入路径

Efficiency=更
少计算+更短
时延+更高信
息利用率

因果 (低时延/流式推理/状态复用)

- 因果建模满足实时部署
- cache memory 复用历史信息
- 避免依赖全局未来帧
- 效率体现为 latency / RTF

数据&先验 (数据仿真/减少无效计算)

- 频带划分聚焦关键语音频段
- 子带建模减少总体计算量
- 先验引导更高效分配计算
- 数据与任务设定影响效率路径



核心判断

高效语音分离系统不是由单一技巧决定的，而是结构设计、融合策略、因果约束与数据质量共同优化的结果。

1. 效率不等于“把模型做小”。

a) 真正有效的方向不是盲目堆深或堆大模型，而是重新设计计算路径和表示方式。

2. 融合方式本身就是效率来源。

a) 早期融合或低层融合往往比后期融合更有效

b) 简单的融合策略有时反而更优，例如 summation 比 concatenation 在性能和计算效率上都更好

c) 复杂 cross-attention 虽然能力强，但参数和 MACs 代价过高，不适合实时应用。

3. 面向部署时，因果性与缓存机制是核心变量。

a) 效率就不能只看参数量，还要看时延、RTF 和历史信息复用能力。

4. 数据与先验决定了效率上限。

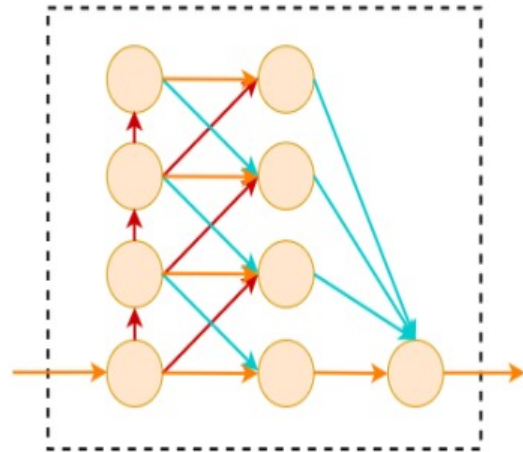
a) 高纯度、语义一致的数据构造可以显著减少训练所需的数据和计算资源

b) 通过多尺度表示、频率/时频先验和信息重组，把计算集中在更关键的区域，从而提升整体效率。

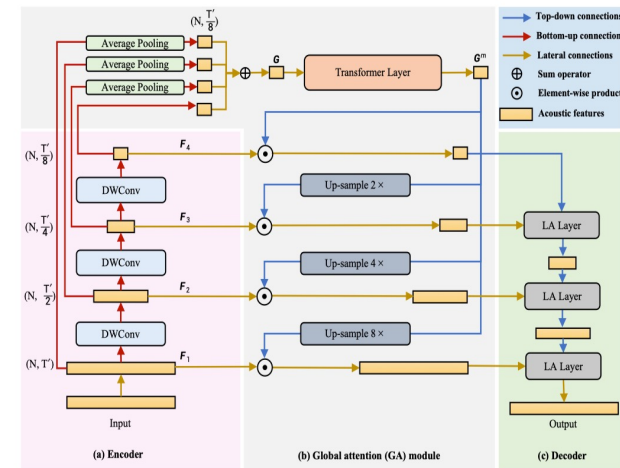
高效架构模块

如何在不明显牺牲性能的情况下重新设计 backbone

A-FRCNN
(NeurIPS 2021)



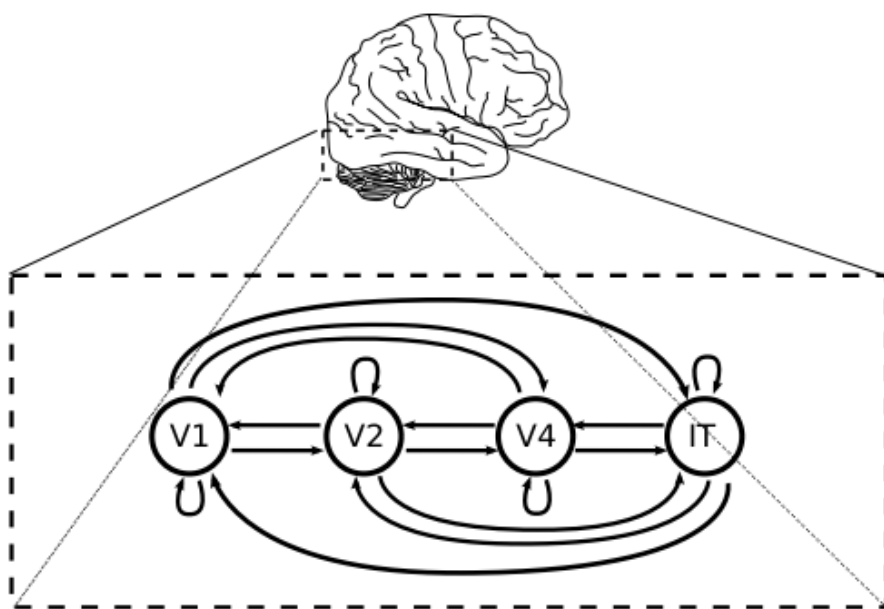
TDANet
(ICLR 2022)



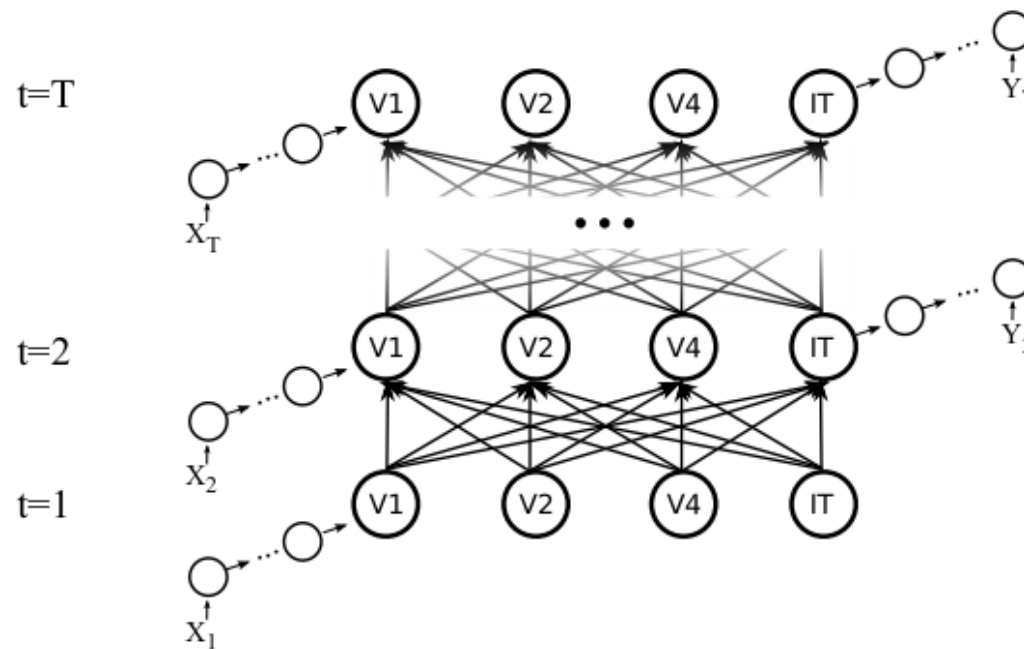
A-FRCNN



- 多尺度融合非常重要，现有方法可能不是最好的方法
- 大脑感觉系统擅长多尺度融合，源于其丰富的自下而上、自上而下和横向连接
- 区域可能不会同时更新状态

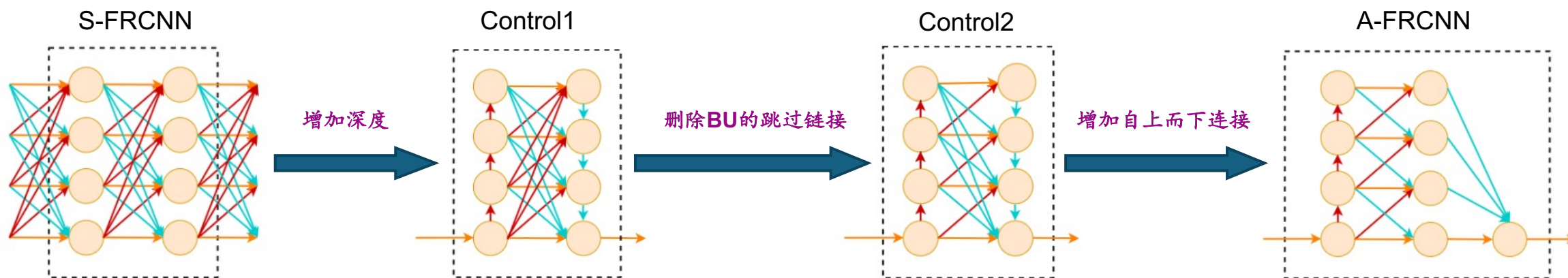


Fully recurrent CNN (FRCNN)



S-FRCNN (Liao and Poggio, 2016)

A-FRCNN



BU: 自下而上; **TD**: 自上而下
更符合生物学原理
长连接比短连接传递信息更慢

A-FRCNN



Model	Si-SNRi (dB)	SDRi (dB)	Params (M)	Time (s)
S-FRCNN	12.1 $\pm$ 0.05	12.5 $\pm$ 0.05	9.6	1.12
Control 1	14.6 $\pm$ 0.04	15.0 $\pm$ 0.03	8.0	0.96
Control 2	14.6 $\pm$ 0.04	15.0 $\pm$ 0.04	6.4	0.81
A-FRCNN (ours)	15.5 $\pm$ 0.04	15.9 $\pm$ 0.04	6.1	0.72
S-FRCNN (light)	11.8 $\pm$ 0.03	12.2 $\pm$ 0.02	6.4	0.97
S-FRCNN (no-skip)	11.4 $\pm$ 0.01	11.9 $\pm$ 0.02	6.4	0.51

S-FRCNN (light), 与 *A-FRCNN* 的参数数量相似。

S-FRCNN (no-skip), 它删除了 *S-FRCNN* 中的所有跳转连接。

- 异步更新方案能以更少的连接获得更好的分离质量。



A-FRCNN — 分离质量

Model	Libri2Mix		WHAM!		WSJ0-2Mix		Params (M)	Time (s)
	SI-SNRi	SDRi	SI-SNRi	SDRi	SI-SNRi	SDRi		
DPCL++	5.9*	6.6*	-	-	10.8	11.2	13.6	0.37
uPIT-BLSTM-ST	7.6*	8.2*	-	-	9.8	10.0	92.7	0.77
Chimera++	6.3*	7.0*	10.0	-	11.5	11.8	32.9	0.66
BLSTM-TasNet	7.9*	8.7*	9.8	-	13.2	13.6	23.6	2.90
Conv-TasNet	12.2	12.7	12.7	-	15.3	15.6	5.6	0.39
Two-Step TDCN	12.0*	12.5*	-	-	16.1	-	8.6	1.85
MSGT-TasNet (light)	-	-	12.3	-	16.8	17.1	37.4	3.37
MSGT-TasNet (dense)	-	-	13.1	-	17.0	17.3	66.8	5.56
SudoRM-RF 0.25x	10.8*	11.3*	10.4*	10.8*	13.4	13.6	0.8	0.24
SudoRM-RF 0.5x	12.2*	12.6*	11.8*	12.2*	15.4	15.6	1.4	0.33
SudoRM-RF 1.0x	13.5*	14.0*	12.9*	13.3*	17.1	17.3	2.7	0.53
SudoRM-RF 2.5x	14.0*	14.4*	13.7*	14.1*	17.4*	17.6*	6.4	1.15
DualPathRNN	14.1*	14.6*	13.7*	14.1*	18.8	19.0	2.7	4.66
Gated DualPathRNN	-	-	15.2	-	20.1	-	7.5	7.35
Sepformer	-	-	-	-	20.4	20.5	26.0	5.22
A-FRCNN-4	12.6 ± 0.01	13.1 ± 0.02	12.0 ± 0.02	12.3 ± 0.02	15.6 ± 0.02	15.8 ± 0.01	6.1	0.33
A-FRCNN-8	15.5 ± 0.04	15.9 ± 0.04	13.4 ± 0.02	13.8 ± 0.02	17.1 ± 0.03	17.3 ± 0.02	6.1	0.72
A-FRCNN-16	16.7 ± 0.03	17.2 ± 0.03	14.5 ± 0.02	14.8 ± 0.03	18.3 ± 0.02	18.6 ± 0.02	6.1	1.51

- A-FRCNN-16 在 Libri2Mix 上表现最佳
- DualPathRNN、Gated DualPathRNN 和 Sepformer 的表现优于 A-FRCNN



A-FRCNN — 分离效率

Model	Training Time (h)
Conv-TasNet	14.16
SudoRM-RF 0.25x	12.50
SudoRM-RF 0.5x	14.20
SudoRM-RF 1.0x	19.65
DualPathRNN	142.51 6天
A-FRCNN-4	14.77
A-FRCNN-8	22.79
A-FRCNN-16	39.44
A-FRCNN-4 (sum)	13.31
A-FRCNN-8 (sum)	20.54
A-FRCNN-16 (sum)	38.33

训练 DualPathRNN 的速度明显慢于其他模型

我们无法训练 Gated DualPathRNN 和 Sepformer, 因为它们需要太大的 GPU 内存

用求和代替连接大大降低了 GFlops

Model	F/B GPU Memory (GB)	F/B GPU Time (ms)	GFlops
Gated DualPathRNN	0.43/7.78	503.8/893.2	125.28
Sepformer	0.62/11.3	311.8/702.1	145.58
SudoRM-RF 2.5x	0.10/4.85	77.2/210.3	19.8
A-FRCNN-16	0.14/4.07	74.4/263.5	123.3
A-FRCNN-16 (sum)	0.13/3.54	70.2/223.3	22.8

A-FRCNN — 样例



混合音频



说话人A



说话人B

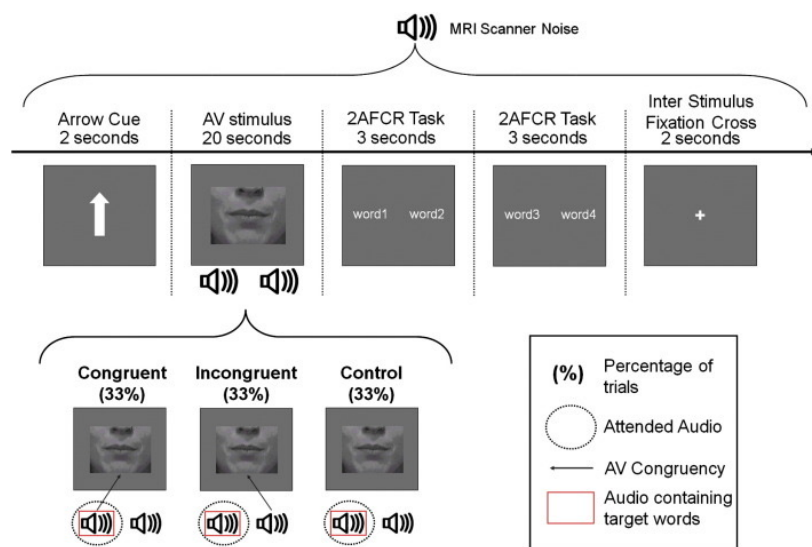


论文: <https://arxiv.org/pdf/2112.02321>

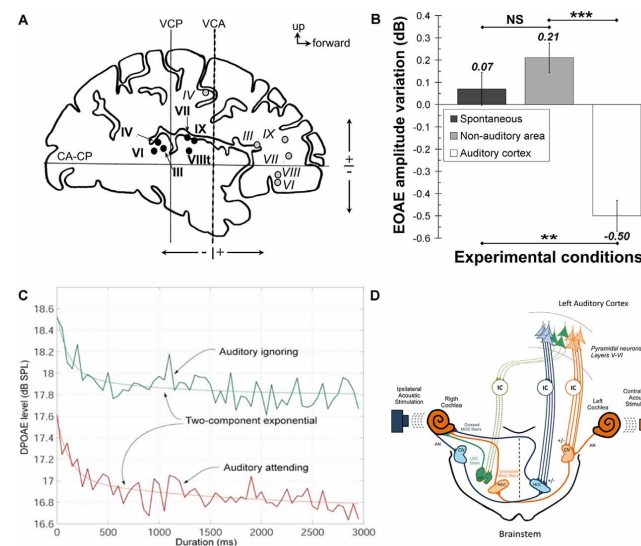
代码: <https://github.com/JusperLee/AFRCNN-For-Speech-Separation>

样例: <https://cslikai.cn/project/AFRCNN>

- 人脑缺乏以**极低功耗**处理大量信息的能力
- **自上而下的注意**是注意与任务相关的刺激并忽略无关干扰的能力的基础
- **注意力机制**如何影响**复杂听觉场景**的处理，目前尚不清楚

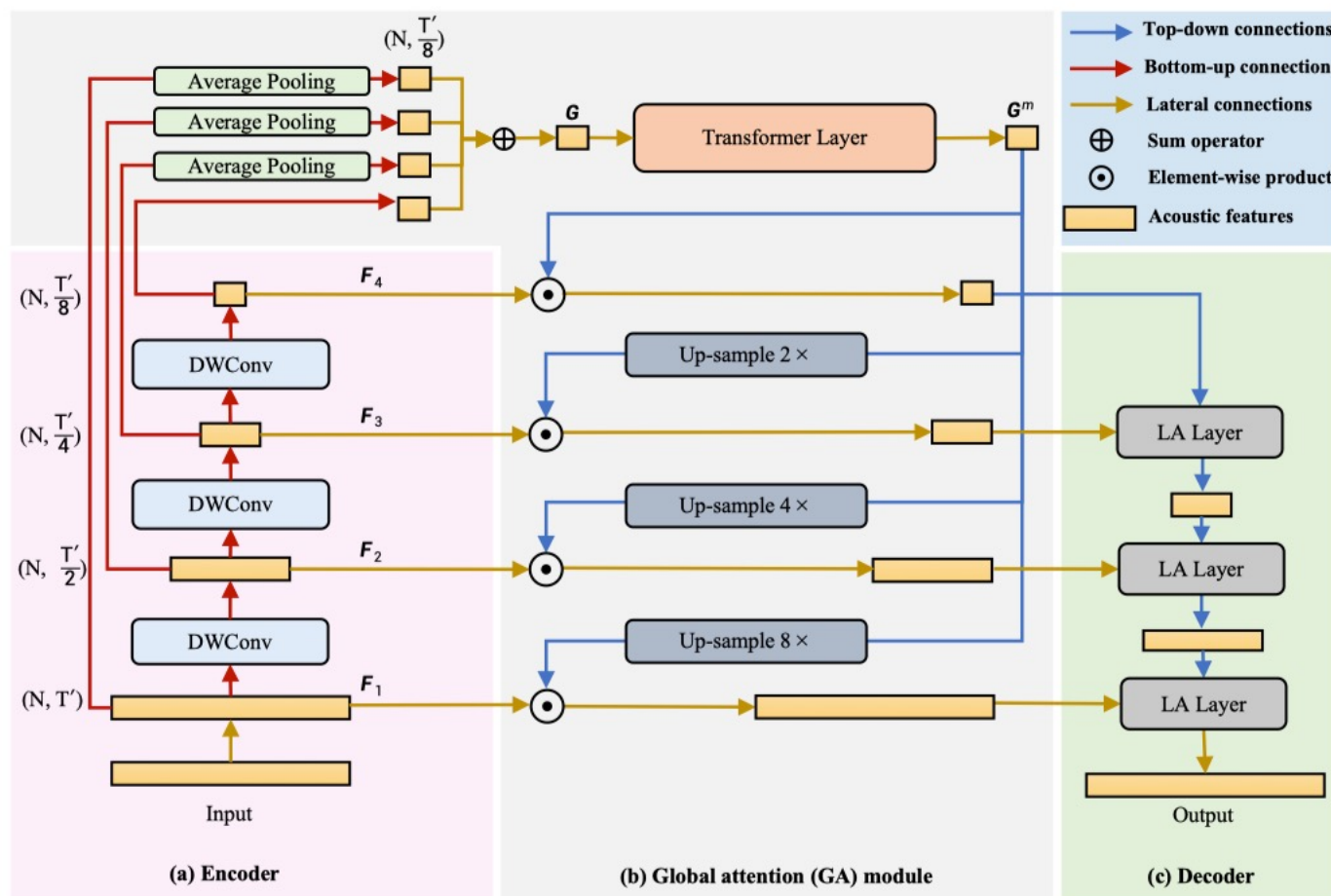


Fernández et al., NeuroImage 2015



Asilador et al., Neural Circuits 2021

- 帶有自上而下注意力的编码器-解码器结构



Model [Authors, Years]	LRS2-2Mix		Libri2Mix		WHAM!		Params (M)
	SI-SNRi	SDRi	SI-SNRi	SDRi	SI-SNRi	SDRi	
BLSTM-TasNet [Luo et al, 2018]	6.1*	6.8*	7.9	8.7	9.8	-	23.6
Conv-TasNet [Luo et al, 2019]	10.6*	11.0*	12.2	12.7	12.7	-	5.6
SuDoRM-RF1.0x [Tzinis et al, 2020]	11.0*	11.4*	13.5	14.0	12.9	13.3	2.7
SuDoRM-RF2.5x [Tzinis et al, 2020]	11.3*	11.7*	14.0	14.4	13.7	14.1	6.4
DualPathRNN [Luo et al, 2020]	12.7*	13.0*	16.1	16.6	13.7	14.1	2.7
DPTNet [Chen et al, 2020]	13.3*	13.6*	16.7	17.1	14.9	15.3	2.7
A-FRCNN-16 [Hu et al, 2021]	13.0*	13.3*	16.7	17.2	14.5	14.8	6.1
Sepformer [Subakan et al, 2021]	13.5*	13.8*	16.5	17.0	14.4	15.0	26.0
TDANet (<i>ours</i>)	13.2	13.5	16.9	17.4	14.8	15.0	2.3
TDANet Large (<i>ours</i>)	14.2	14.5	17.4	17.9	15.2	15.4	2.3

Model	F GPU RTF (ms)	B GPU RTF (ms)	CPU RTF (s)	MACs (G/s)
BLSTM-TasNet	233.85	654.14	5.9	43.0
Conv-TasNet	15.28	56.91	0.82	10.2
SuDoRM-RF 1.0x	27.86	95.37	0.75	4.6
SuDoRM-RF 2.5x	64.70	228.57	1.73	10.1
DualPathRNN	88.19	241.54	8.13	85.3
DPTNet	103.26	689.06	10.49	102.5
A-FRCNN-16	61.16	183.65	5.32	125.3
Sepformer	65.61	184.91	7.55	86.9
TDANet (<i>ours</i>)	11.76	86.56	0.79	4.7
TDANet Large (<i>ours</i>)	23.77	97.92	1.78	9.1

- TDANet 实现了最先进分离性能
- 与 DPTNet 和 Sepformer 相比, TDANet 的 MAC 是它们的 5%, 而训练时间分别是它们的 13% 和 47%
- 与 A-FRCNN 和 SuDORM-RF 2.5x 相比, TDANet 的 GPU 推理时间分别是它们的 19% 和 18%, GPU 训练时间分别是它们的 47% 和 37%, CPU 推理时间分别是它们的 15% 和 46%。



A-FRCNN & TDANet

A-FRCNN 强调异步递归更新；TDANet 强调 top-down attention。共同点：二者都在重构信息流，而不只是单纯放大模型。

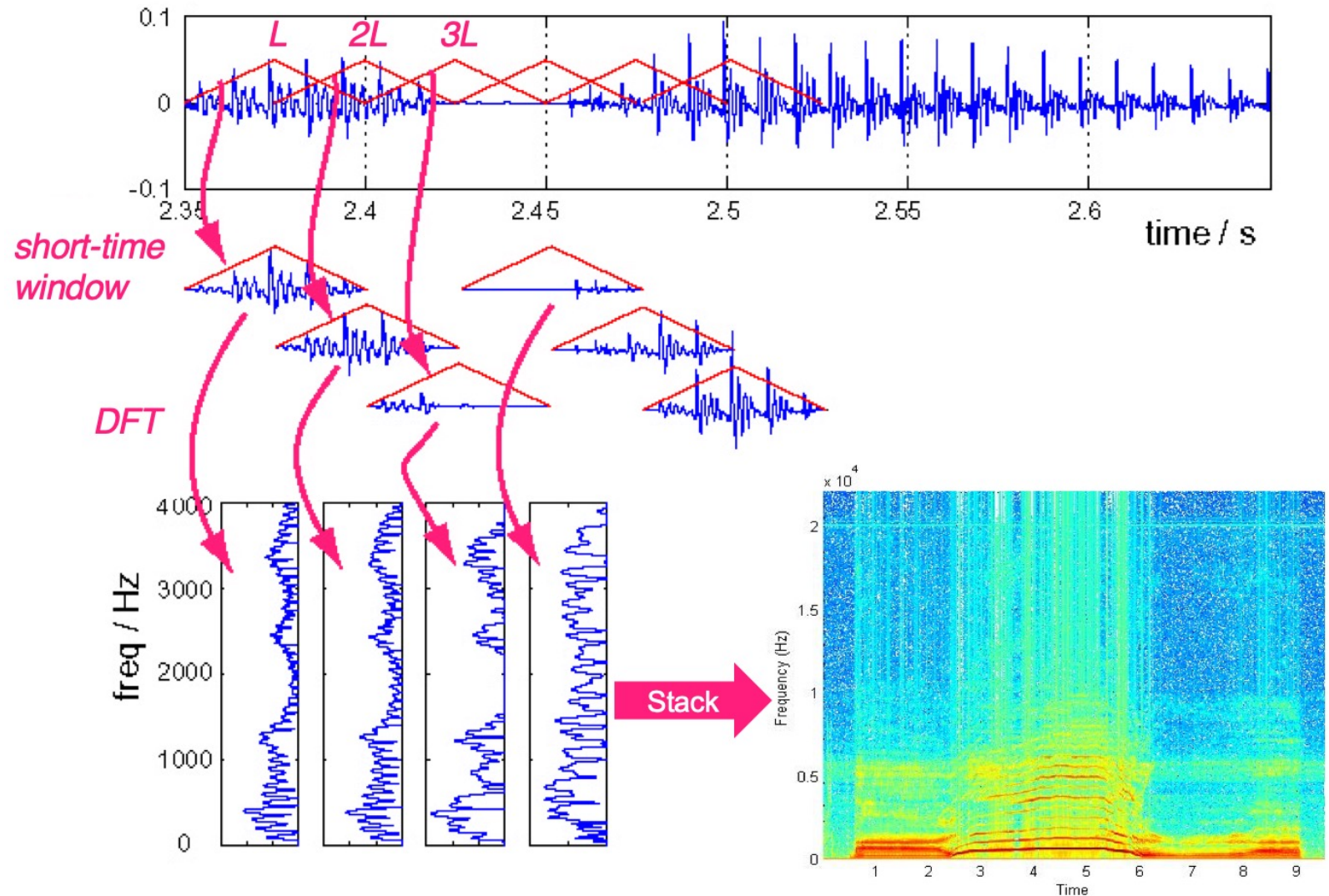
维度	A-FRCNN	TDANet
核心理念	异步 fully recurrent 更新，重排多尺度特征传播顺序。	高效 encoder-decoder + top-down attention，在低复杂度下保持性能。
信息流	强调“何时融合”——通过递归与延迟更新组织信息流。	强调“如何调制”——通过 top-down attention 让高层信息反向作用于低层特征。
效率启发	效率来自传播路径重排，而不只是减参数。	效率来自轻量结构设计与更高效的特征调制。
局限	递归结构清晰，但复杂度控制仍依赖具体实现。	attention 设计更复杂，收益依赖具体模块设计。
一句话	改变信息什么时候流动。	改变信息如何被调制。

局部总结： A-FRCNN 与 TDANet 的共同点，不是都“更小”，而是都把效率建立在信息流重构上：前者偏重递归更新顺序，后者偏重 top-down 调制机制。



为什么要回到时频域

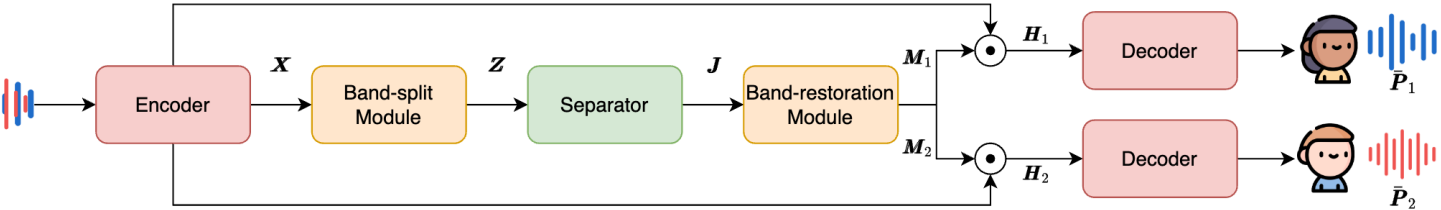
- ✓ 时域方法在声学先验利用上可能不够充分。
- ✓ 时频域显式保留谱结构，更容易做拆维建模。





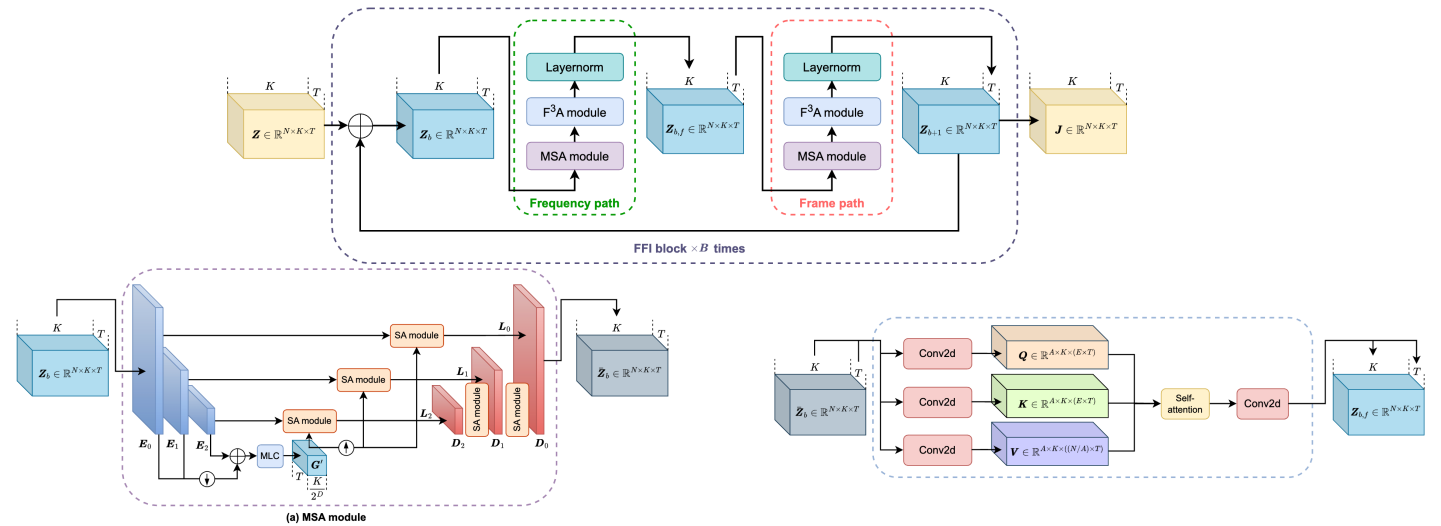
TIGER: 频带压缩与交错重建

1. 利用频带划分与压缩先验。
2. 通过 gain extraction and reconstruction 降低计算。
3. 94.3% 参数下降、95.3% MACs 下降



Methods	Libri2Mix		LRS2-2Mix		EchoSet	
	SDRi	SI-SDRi	SDRi	SI-SDRi	SDRi	SI-SDRi
Conv-TasNet (Luo & Mesgarani, 2019)	12.50	12.10	11.0	10.6	7.69	6.89
DualPathRNN (Luo et al., 2020)	11.64	11.26	13.0	12.7	5.87	5.06
SudoRM-RF1.0x (Tzinis et al., 2020)	13.58	13.16	11.4	11.0	7.70	6.84
A-FRCNN-16 (Hu et al., 2021)	16.73	16.32	13.3	13.0	9.64	8.76
TDANet Large (Li et al., 2023)	16.11	15.64	14.5	14.2	10.14	9.21
BSRNN (Luo & Yu, 2023)	17.38	16.96	14.4	14.1	12.75	12.23
TF-GridNet (Wang et al., 2023)	19.56	19.24	15.7	15.4	13.73	12.85
TIGER (small)	17.09	16.67	14.2	13.9	13.15	12.58
TIGER (large)	18.34	17.97	15.3	15.1	14.22	13.73

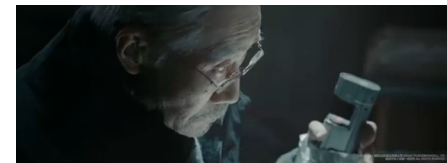
Methods	Paras (M)	MACs (G/s)	Training		CPU Time	Inference	
			GPU Time	GPU Memory		GPU Time	GPU Memory
Conv-TasNet 2019	5.62	7.19	92.96	1436.94	64.21	13.17	28.78
DualPathRNN 2020	2.72	45.01	67.23	1813.55	723.13	30.38	298.09
SudoRM-RF1.0x 2020	2.72	4.65	118.46	1353.43	104.32	20.66	24.42
A-FRCNN-16 2021	6.13	81.28	230.53	2925.83	478.58	82.65	163.82
TDANet Large 2023	2.33	9.19	263.43	4260.36	434.44	74.27	136.96
BSRNN 2023	25.97	98.70	258.55	1093.11	897.27	78.27	130.24
TF-GridNet 2023	14.43	323.75	284.17	5432.94	2019.60	94.30	491.73
TIGER (small)	0.82	7.65	160.17	2049.46	351.15	42.38	122.23
TIGER (large)	0.82	15.27	229.23	3989.59	765.47	74.51	122.23



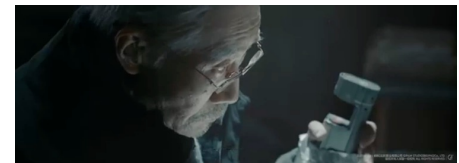
混合音频



人声



音乐



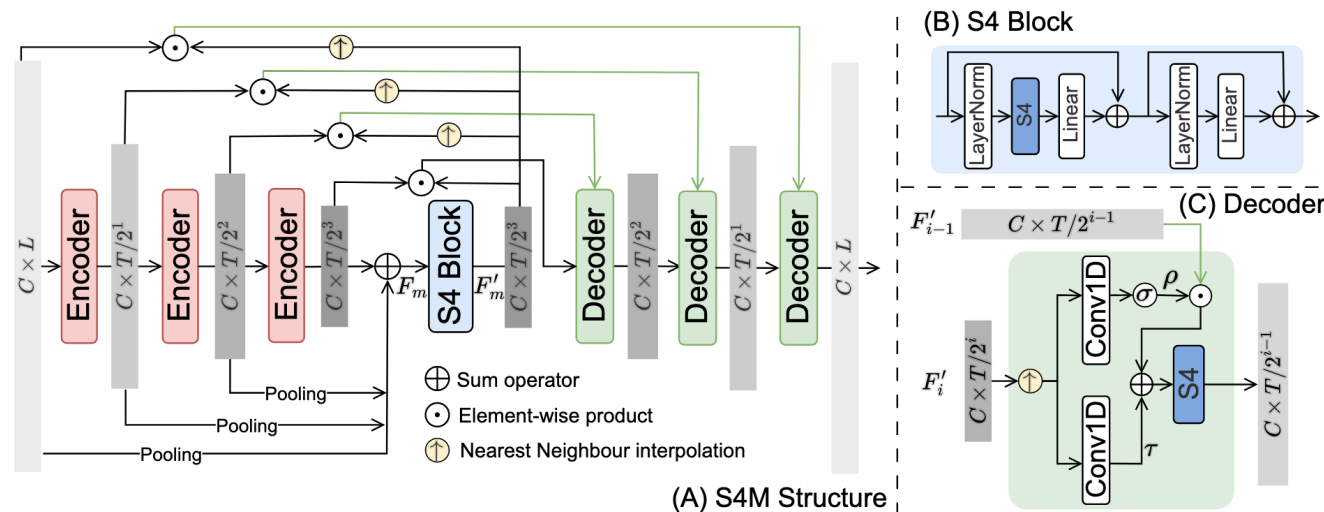
音效

Method	SI-SDRi (dB)	SDRi (dB)	MACs (G)	Params (K)	CPU Time (ms)	GPU Time (ms)	GPU Memory (MB)
SudoRM-RF GC3	4.6	5.0	0.81/4.84	303.57	123.67/612.38	32.68/58.35	25.20/151.25
TIGER Tiny	10.4	10.7	0.72/7.48	102.12	219.89/867.61	30.21/55.70	25.44/153.12



S4M: 状态空间模型进入语音分离

- ✓ S4M 基于 neural state-space model。
- ✓ 其目标是高效捕获长序列依赖。



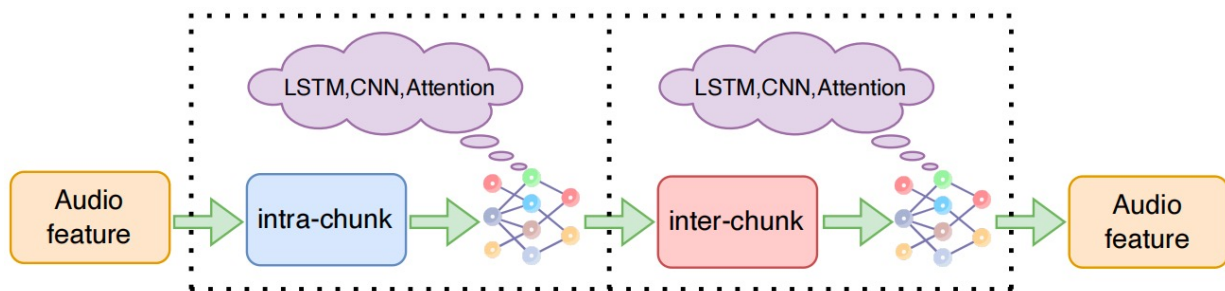
Model	GPU-RTF- <i>f</i> (ms)	GPU-RTF- <i>b</i> (ms)	CPU-RTF- <i>f</i> (s)	MACs (G/s)
BLSTM-TasNet	233.85	654.14	5.90	43.0
SuDoRM-RF	64.70	228.57	1.73	10.1
DPRNN	88.79	241.54	8.13	85.3
A-FRCNN	61.16	183.65	5.32	125.3
TDANet	23.77	97.92	1.78	9.1
TDANet (own)	61.25	368.54	5.97	9.1
Sepformer	65.61	184.91	7.55	86.9
TF-GridNet	100.52	285.37	86.4	128.7
S4M-tiny	18.19	73.62	1.34	8.0
S4M	40.15	132.83	2.57	38.7

Model	LibriMix		LRS2-Mix		# Para. (M)
	SI-SDRi	SDRi	SI-SNRi	SDRi	
BLSTM-TasNet	7.9	8.7	6.1	6.8	23.6
Conv-TasNet	12.2	12.7	10.6	11.0	5.6
DPRNN	16.1	16.6	12.7	13.0	2.7
SuDoRM-RF	14.0	14.4	11.3	11.7	6.4
Sepformer	16.5	17.0	13.5	13.8	26.0
WaveSplit	16.6	17.2	13.1	13.4	29.0
A-FRCNN	16.7	17.2	13.0	13.3	6.1
TDANet	17.4	17.9	14.2	14.5	2.3
S4M-tiny	16.2	16.6	14.2	14.5	1.8
S4M	16.9	17.4	15.3	15.5	3.6

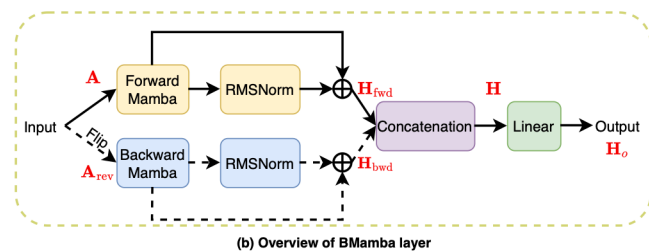
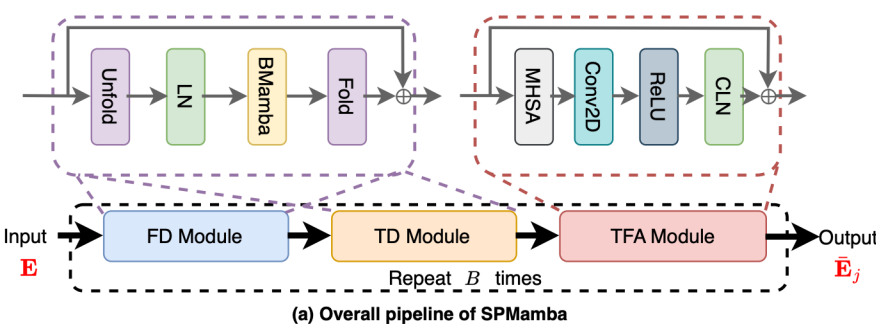


SPMamba: Mamba长序列建模

- ✓ CNN 模型：感受野有限，难以捕获完整音频上下文。
- ✓ RNN/LSTM 模型：短期记忆较强，但对长距离依赖建模效率较低。
- ✓ Transformer 模型：计算复杂度较高，在长序列场景下开销较大。



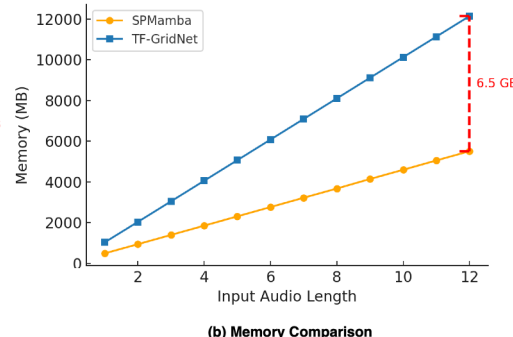
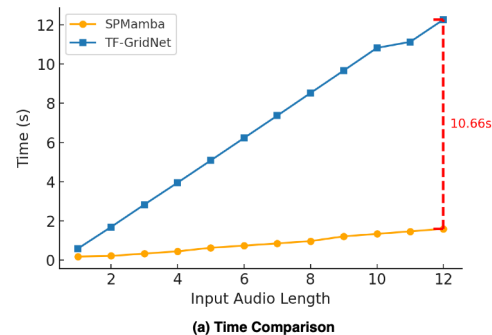
- **Mamba 基于状态空间模型 (SSM)**，适合序列建模任务。
- 相比 Transformer，能够以**更低复杂度**处理长序列。
- 相比 CNN/RNN，具有更强的**长距离依赖建模能力**。
- 下图表明：随着音频长度增加，**SPMamba 的性能下降更缓慢**，体现出更好的长序列鲁棒性。



Model	WSJ0-2Mix		Libri2Mix		Params(M)	Macs (G/s)
	SDRi	SI-SNRi	SDRi	SI-SNRi		
Conv-TasNet [1]	15.6	15.3	12.7	12.2	5.62	10.23
DualPathRNN [7]	19.0	18.8	16.6	16.1	2.72	85.32
SudoRM-RF [2]	19.1	18.9	14.4	14.0	2.72	4.60
A-FRCNN [4]	18.6	18.3	17.2	16.7	6.13	81.20
TDANet [3]	18.7	18.5	17.9	17.4	2.33	9.13
BSRNN [26]	20.9	20.7	15.7	15.2	25.97	98.69
TF-GridNet [10]	23.1	22.8	20.1*	19.8*	14.43	445.56
SPMamba (Ours)	22.7	22.5	20.4	19.9	6.14	238.69

Model	WHAM!		WHAMR!		Echo2Mix	
	SDRi	SI-SNRi	SDRi	SI-SNRi	SDRi	SI-SNRi
Conv-TasNet [1]	13.0	12.7	-	8.3	7.7	6.9
DualPathRNN [7]	14.1	13.7	-	10.3	5.9	5.1
SudoRM-RF [2]	14.1	13.7	-	-	7.7	6.8
A-FRCNN [4]	14.8	14.5	-	-	9.6	8.8
TDANet [3]	15.4	15.2	-	-	10.1	9.2
BSRNN [26]	16.5	16.2	-	-	12.8	12.2
TF-GridNet [10]	17.2*	16.9*	14.9	16.4	13.7*	12.8*
SPMamba (Ours)	17.6	17.4	15.2	16.6	16.1	15.3

模型性能





S4M 与 SPMamba 对比

两者都在用状态空间模型解决长序列 + 线性复杂度问题；但 S4M 更偏原生的时域 SSM 分离框架，SPMamba 更偏在 TF-GridNet 上落地双向 Mamba 的系统升级。

维度	S4M	SPMamba
共同目标	线性复杂度 用状态空间模型建模长程依赖，提升长语音序列下的分离效率。	线性复杂度 用状态空间模型处理长音频，在更低复杂度下捕获长距离上下文。
系统框架	框架型设计 更偏“从 SSM 出发构建分离网络”，整体思路更原生。	骨架增强型 基于 TF-GridNet，把原有 BLSTM 模块替换为 bidirectional Mamba。
状态空间实现	以 S4 / Structured SSM 为代表，更强调结构化长程依赖建模。	以 Mamba / Selective SSM 为代表，并进一步设计成 BMamba。
参数量	代表配置约 3.59M，整体取向偏轻量。	论文主结果 6.14M；MACs 为 238.69 G/s。
任务域	主要面向 时域语音分离，强调长序列建模能力。	主要面向 时频域语音分离；数据集包括 WSJ0-2Mix、WHAM!、Libri2Mix、Echo2Mix。
是否双向	双向不是主要卖点，重点在于 S4 作为核心建模框架。	是。BMamba 同时进行 forward / backward 建模。



deep mask estimation 的角色

高效化不一定只来自 backbone 重设计；也可能来自 mask estimation module 的重新设计。更深的 mask estimation，本质上可以看作对 overseparation-grouping 思路的高效近似。



高效融合模块

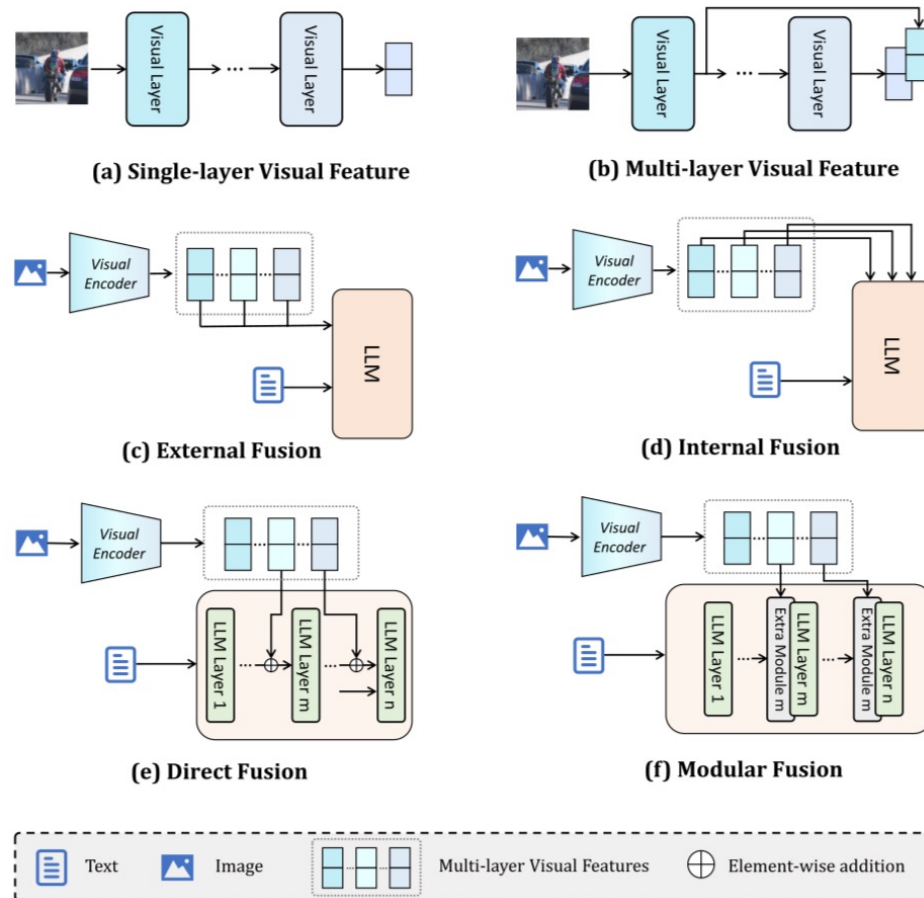
核心问题：问题不只是“是否加入视频”，而是“如何轻量地用好视频”。视觉线索提升了分离鲁棒性，但**高效融合**才是真正的挑战所在。

为什么需要视觉线索？

- 纯音频分离在强噪声、混响、多人同时说话的场景中性能显著下降。视觉线索（如唇形运动）不受声学环境影响，可提供额外说话人信息。

加入视频的代价

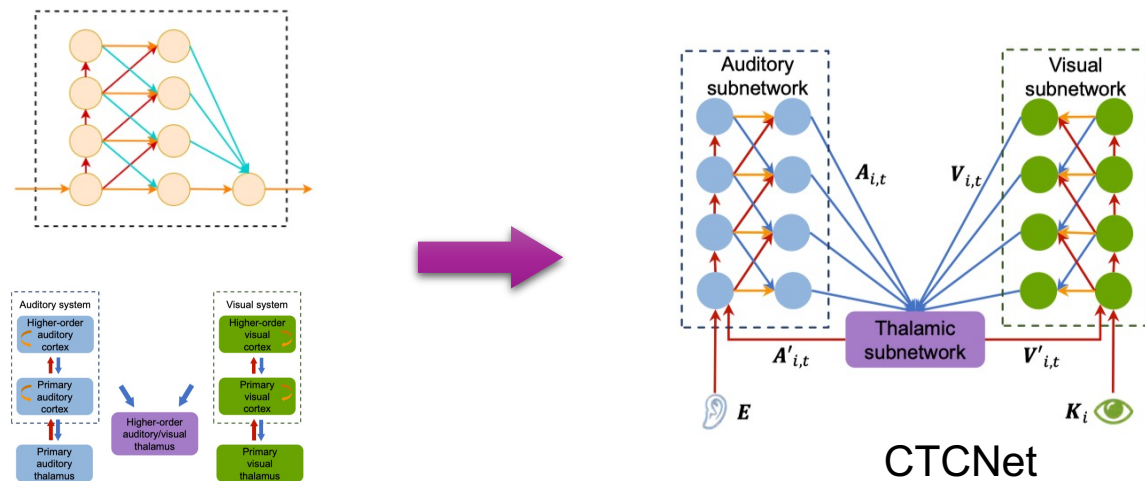
- 许多方法虽然性能强，但模型参数量大、计算代价高，难以作为预处理步骤部署在实际应用中。
- 大型视觉编码器（如 3D ResNet）语义表达好但代价高；轻量设计则难以提取有效语义，压缩后分离性能明显下滑。
- 许多方法只在单一时间尺度上融合特征，限制了对多模态信息的充分利用；跨模态融合的效率至今仍是开放挑战。





CTCNet: 脑启发的音视频分离

模仿皮质—丘脑—皮质回路：先分别学习音频与视觉的层级表示，再通过类丘脑模块做跨模态融合并反馈，以**更少参数**大幅超越现有 AVSS 方法。



1 自底向上：分别学习层级表示

听觉子网络与视觉子网络各自以 bottom-up 方式提取多尺度特征，模拟听觉皮层与视觉皮层的分层感知机制。

2 丘脑融合：跨模态信息整合

丘脑子网络接收来自两个子网络所有层的投影，融合音频与视觉特征（类比高阶感觉丘脑的多模态整合功能）。

3 自顶向下反馈：循环迭代

融合后的特征反馈回听觉与视觉子网络，上述过程重复多次，最终只保留听觉子网络继续循环以减少计算量。

性能对比（论文 Table 7，三个数据集）

SI-SNRi / SDRi (dB), ↑越高越好

方法	LRS2-2Mix SI-SNRi	LRS2-2Mix SDRi	LRS3-2Mix SI-SNRi	LRS3-2Mix SDRi	VoxCeleb2 SI-SNRi	VoxCeleb2 SDRi	Params (M)
AOSS Methods							
Conv-TasNet	10.3	10.7	11.1	11.4	6.9	7.5	5.6
A-FRCNN	9.4	10.1	12.5	12.8	7.8	8.2	6.3
AVSS Methods							
AV-ConvTasNet	12.5	12.8	11.2	11.7	9.2	9.8	16.5
VisualVoice	11.5	11.8	9.9	10.3	9.3	10.2	77.8
AVLIT-8	12.8	13.1	13.5	13.6	9.4	9.9	5.6
CTCNet (本文)	14.3	14.6	17.4	17.5	11.9	13.1	7.0

绿色高亮行：CTCNet 以 7.0M 参数在三个数据集上全面超越 VisualVoice (77.8M) 等现有方法，在 LRS3-2Mix 上 SI-SNRi 达 17.4 dB，较 AVLIT-8 高出约 3.9 dB。

控制组对比（论文 Table 6，LRS2-2Mix）

模型	SDRi (dB)	SI-SNRi (dB)	Params (M)
DFTNet (无丘脑)	13.1	12.8	3.6
CCNet (皮层直连)	10.9	10.4	18.6
CACNet (高层融合)	10.2	9.9	7.1
CTCNet (丘脑融合)	13.5	13.2	7.0

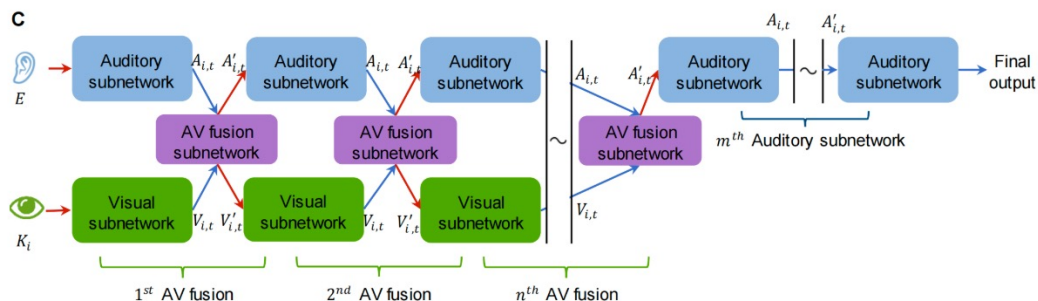


CTCNet: 脑启发的音视频分离

关键不是“**像脑**”，而是通过**递归反馈**让多模态互相调制：丘脑结构为音视频特征提供了**跨尺度、跨模态的持续迭代通道**，这才是性能提升的真正原因。

Figure C: 多轮 AV 融合递归示意

来源: 论文 Figure 1C · $1^{\text{st}} \rightarrow n^{\text{th}}$ AV fusion 迭代过程



四个关键机制解析

● MECHANISM 1

多尺度特征汇聚

丘脑子网络同时接收听觉与视觉子网络所有层的投影，而非只融合顶层特征（如 CACNet）。这使得低层细节与高层语义均参与跨模态融合。

● MECHANISM 2

递归反馈调制

融合结果反馈回两个子网络，并重复多次（n 个融合周期）。每一次迭代都让音频特征被视觉调制、视觉特征被音频调制，实现持续的双向校正。

● MECHANISM 3

底层-顶层-同层三向连接

各子网络内部同时保留 bottom-up、top-down、lateral 三种连接，使每层特征都能从相邻层获取信息，类似于皮层内部的局部感知整合机制。

● MECHANISM 4

轻量单层丘脑结构

丘脑子网络本身只有单层结构，但因为汇聚了全局多尺度信息，其表达能力远超参数量更大的直接跨模态连接（如 CCNet 的 18.6M 参数）。

常见误解 vs 实质

✗ 误解

CTCNet 有效是因为它“模仿了大脑结构”，生物合理性本身带来了性能提升。



✓ 实质

生物启发提供了一种架构归纳偏置：丘脑模块实现了多尺度特征的跨模态汇聚与反馈，这是其他控制模型（CCNet、CACNet）所缺少的。

CTCNet: 脑启发的音视频分离



混合音频



女说话人

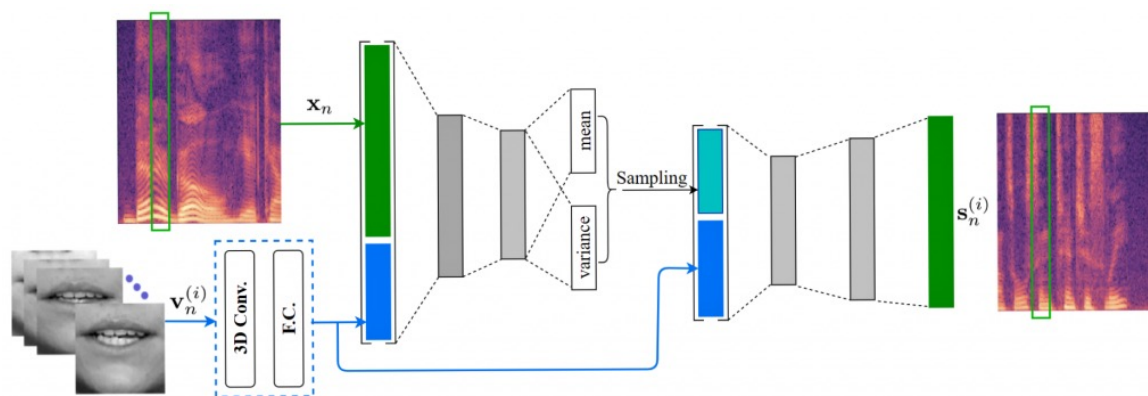


男说话人

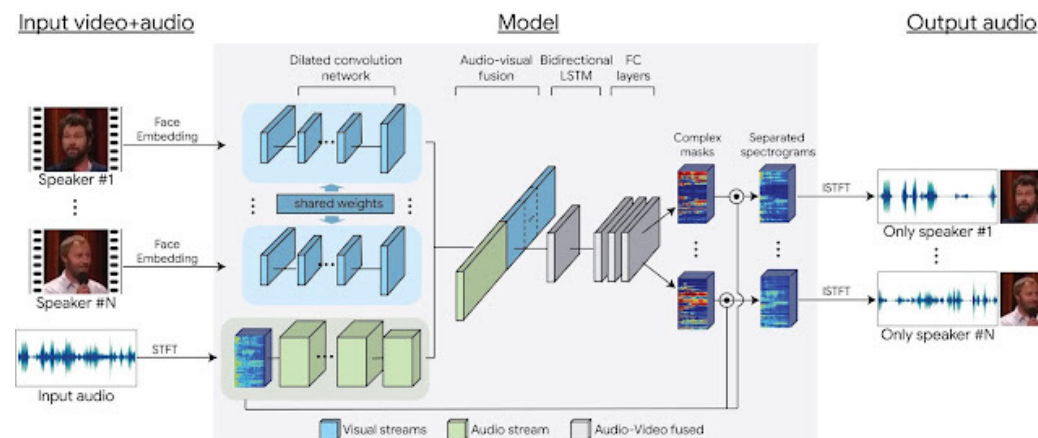


IIANet: 模态内与模态间的选择性注意

- 现有的视听语音分离方法大多只在粗粒度或者细粒度上进行音视频信息融合
- 许多视听语音分离模型在跨模态融合过程中采用注意力机制；一些方法使用相似矩阵来计算注意力，这大大增加了计算复杂度



<https://team.inria.fr/perception/research/avss/>

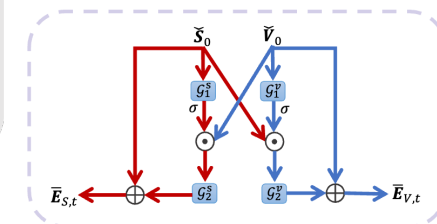
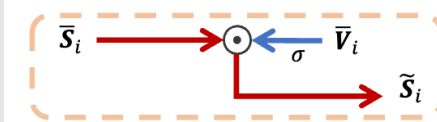
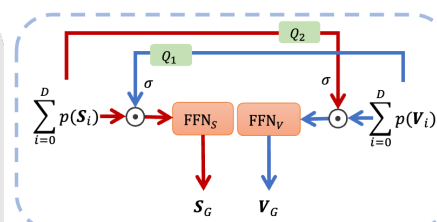
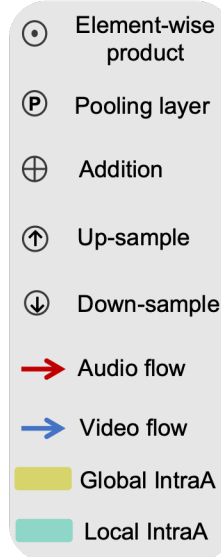
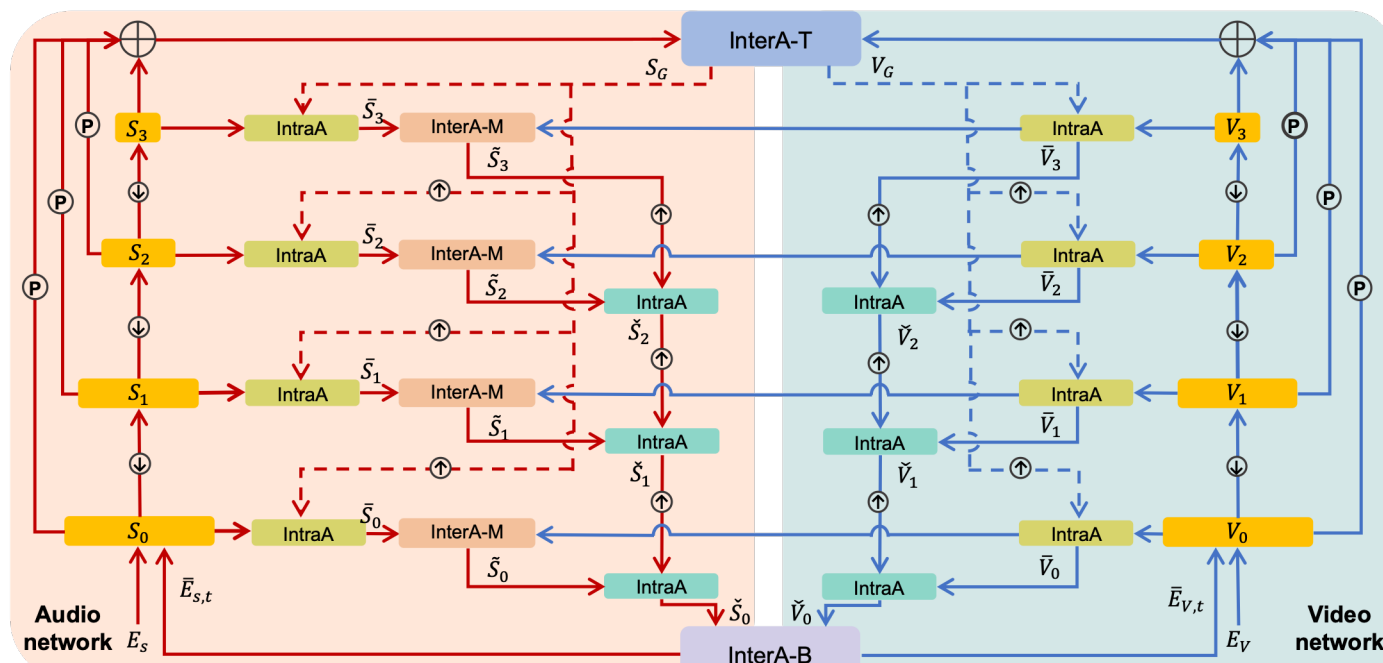


<https://research.google/blog/looking-to-listen-audio-visual-speech-separation/>



IIANet: 模态内与模态间的选择性注意

- IntraA: 负责模态内语义建模。通过 top-down selective attention, 在音频或视觉内部聚焦更相关的局部与全局信息。
- InterA: 负责模态间语义交互。用视觉线索调制音频特征, 在不同层级提取与目标说话人一致的跨模态信息。





IIANet: 模态内与模态间的选择性注意

- 更好的分离质量（在分离质量上优于AVSS方法至少**1.7 dB SI-SNRi**）

Methods	LRS2			LRS3			VoxCeleb2		
	SI-SNRi	SDRi	PESQ	SI-SNRi	SDRi	PESQ	SI-SNRi	SDRi	PESQ
AVConvTasNet (Wu et al., 2019)	12.5	12.8	2.69	11.2	11.7	2.58	9.2	9.8	2.17
LWTNet (Afouras et al., 2020)	-	10.8	-	-	4.8	-	-	-	-
VisualVoice (Gao & Grauman, 2021)	11.5	11.8	2.78	9.9	10.3	2.13	9.3	10.2	2.45
CaffNet-C (Lee et al., 2021)	-	10.0	1.15	-	9.8	-	-	7.6	-
AVLiT-8 (Martel et al., 2023)	12.8	13.1	2.56	13.5	13.6	2.78	9.4	9.9	2.23
CTCNet (Li et al., 2022)	14.3	14.6	3.08	17.4	17.5	3.24	11.9	13.1	3.00
IIANet (ours)	16.0	16.2	3.23	18.3	18.5	3.28	13.6	14.3	3.12
IIANet-fast (ours)	<u>15.1</u>	<u>15.4</u>	<u>3.11</u>	<u>17.8</u>	<u>17.9</u>	<u>3.25</u>	<u>12.6</u>	<u>13.6</u>	<u>3.01</u>

- 更高效的分离（IIANet可以使用**11%的MAC**、**44%的参数**和**16%的GPU显存**来实现更高的分离质量）

Methods	Computation Cost		Inference Time		GPU Memory (MB)
	MACs (G)	Params (M)	CPU (s)	GPU (ms)	
AVConvTasNet (Wu et al., 2019)	23.8	16.5	1.06	<u>62.51</u>	117.05
VisualVoice (Gao & Grauman, 2021)	9.7	77.8	2.98	110.31	313.74
AVLiT (Martel et al., 2023)	18.2	<u>5.8</u>	0.46	62.51	<u>24.0</u>
CTCNet (Li et al., 2022)	167.1	7.0	1.26	84.17	75.8
IIANet (ours)	18.6	3.1	1.27	110.11	12.5
IIANet-fast (ours)	<u>11.9</u>	3.1	<u>0.52</u>	70.94	12.5



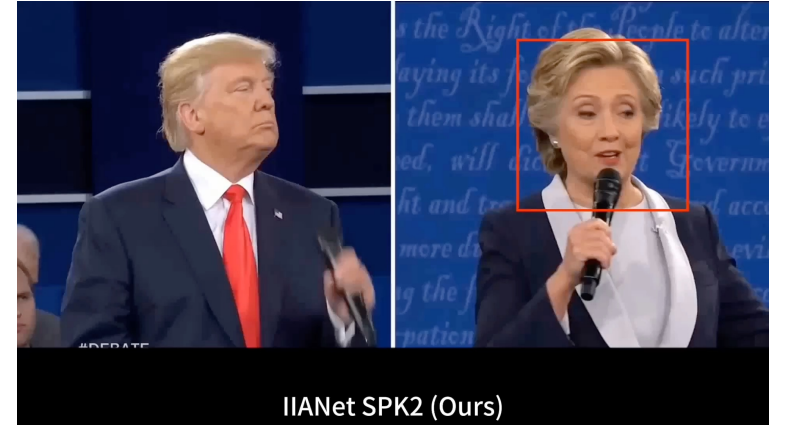
IIANet: 模态内与模态间的选择性注意



混合音频



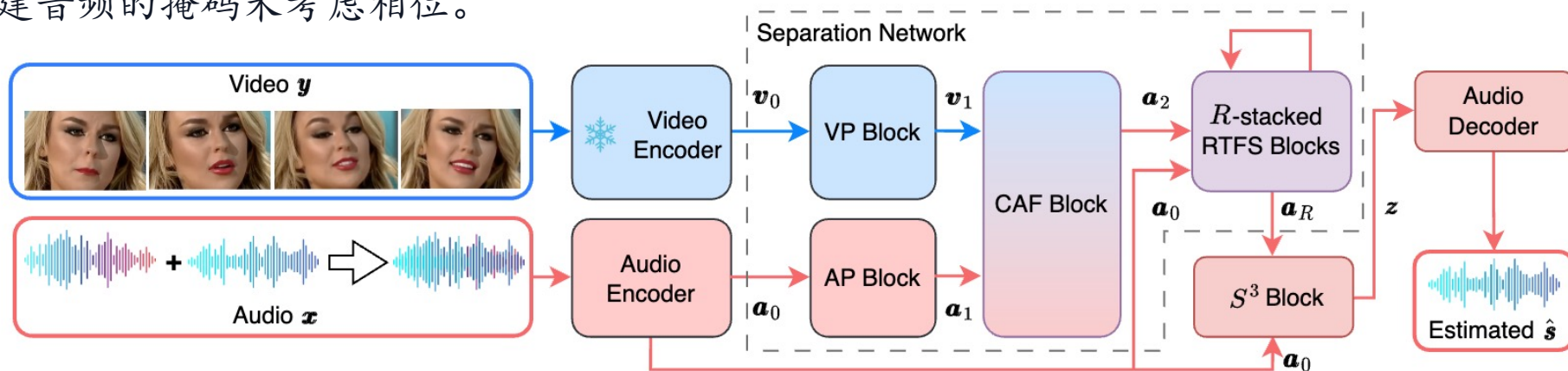
川普



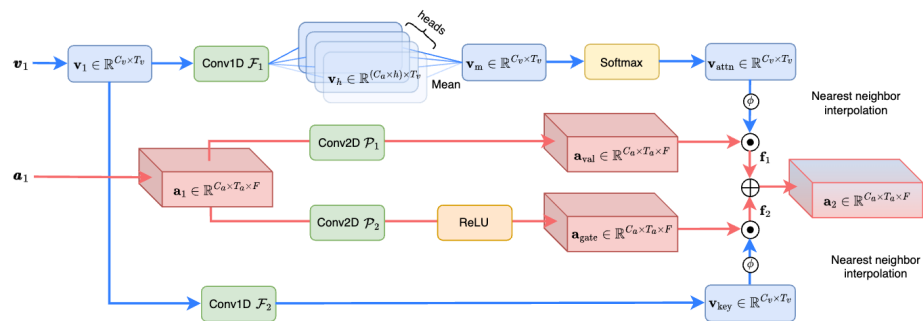
希拉里

RTFS-Net: 极度轻量化的多模态融合

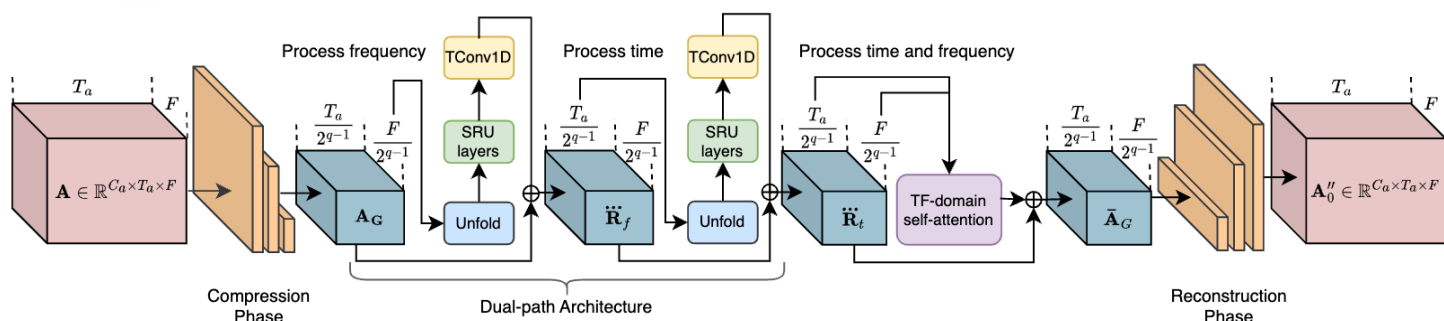
- ✓ 时域方法效率较低
- ✓ 以往方法中时频域方法的性能往往较差。
 1. 没有对时间和频率进行显式建模
 2. 重建音频的掩码未考虑相位。



RTFSNet结构图



CAF Block结构图



RTFSNet Block结构图



RTFS-Net: 极度轻量化的多模态融合

AV Fusion Strategies	LRS2-2Mix		RTFS-Net		Fusion	
	SI-SNRi	SDRi	Params (K)	MACs (G)	Params (K)	MACs (M)
CTCNet Fusion (adapted)	11.3	11.7	528	14.3	197	6365
CAF Block (ours)	11.7	12.1	339	8.0	7	83

融合模块复杂度从 197K / 6365M 大幅降到 7K / 83M

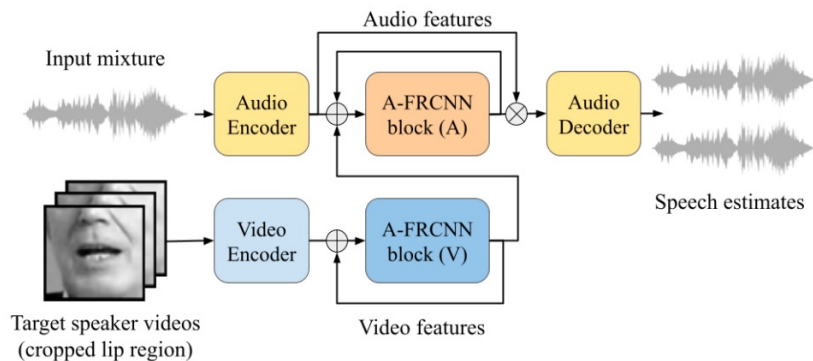
Model	LRS2-2Mix		LRS3-2Mix		VoxCeleb2-2Mix		Params (M)	MACs (G)	Time (ms)
	SI-SNRi	SDRi	SI-SNRi	SDRi	SI-SNRi	SDRi			
CaffNet-C*	-	12.5	-	12.3	-	-	-	-	-
Thanh-Dat	-	11.6	-	-	-	-	-	-	-
AV-ConvTasnet	12.5	12.8	11.2	11.7	9.2	9.8	16.5	-	60.3
VisualVoice	11.5	11.8	9.9	10.3	9.3	10.2	77.8	-	130.2
AVLIT	12.8	13.1	13.5	13.6	9.4	9.9	5.8	36.4	53.4
CTCNet	14.3	14.6	17.4	17.5	11.9	13.1	7.1	167.2	122.7
RTFS-Net-4	14.1	14.3	15.5	15.6	11.5	12.4	0.7	21.9	57.8
RTFS-Net-6	14.6	14.8	16.9	17.1	11.8	12.8	0.7	30.5	64.7
RTFS-Net-12	14.9	15.1	17.5	17.6	12.4	13.6	0.7	56.4	109.9

RTFS-Net-12 仅用 0.7M 参数和 56.4G MACs, 即显著低于 CTCNet 的 7.1M 和 167.2G, 同时推理速度也更快。

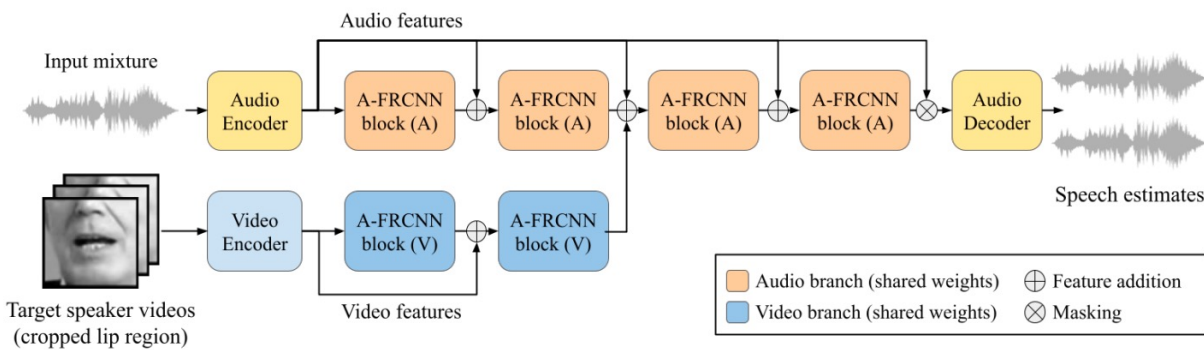


AVLIT: 在 noisy 环境中的轻量迭代模型

基于 A-FRCNN 构建音频+视频双分支，结合迭代块权重共享与 Progressive Learning，以 5.75M 参数、23.97MB 推理内存实现 SOTA 分离性能。



(A) Folded view of AVLIT.



(B) Unfolded view of AVLIT with $N_A = 4$, $N_V = 2$, and $P = \{2\}$.

Model	Inputs		NTCD-TIMIT			LRS3+WHAM!		
	A	V	PESQ $\uparrow$	ESTOI $\uparrow$	SI-SDRi $\uparrow$	PESQ $\uparrow$	ESTOI $\uparrow$	SI-SDRi $\uparrow$
Unprocessed	—	—	1.19	0.33	—	1.05	0.34	—
ConvTasNet [3]	✓	×	1.35	0.38	8.76	1.24	0.51	9.66
DPRNN [23]	✓	×	1.32	0.39	9.31	1.40	0.45	10.93
A-FRCNN-2 [17]	✓	×	1.22	0.21	5.94	1.18	0.46	6.48
A-FRCNN-4 [17]	✓	×	1.24	0.26	6.61	1.23	0.46	7.32
A-FRCNN-8 [17]	✓	×	1.30	0.31	6.92	1.25	0.49	9.21
AVConvTasNet [24]	✓	✓	1.33	0.40	9.02	1.29	0.60	6.21
LAVSE [25]	✓	✓	1.31	0.37	6.22	1.24	0.50	5.59
L2L [26]	✓	✓	1.23	0.26	3.36	1.16	0.51	7.60
VisualVoice [27]	✓	✓	1.45	0.43	10.04	1.48	0.63	11.87
AVLIT-2 (Ours)	✓	✓	1.36	0.40	9.05	1.30	0.55	9.10
AVLIT-4 (Ours)	✓	✓	1.38	0.42	10.15	1.41	0.62	10.99
AVLIT-8 (Ours)	✓	✓	1.43	0.45	11.00	1.52	0.68	12.42

Model	Inputs		Model complexity		Inference time		Memory footprint	
	A	V	MACs (G) $\downarrow$	Params (M) $\downarrow$	CPU (s) $\downarrow$	GPU (s) $\downarrow$	Train (MB) $\downarrow$	Inference (MB) $\downarrow$
ConvTasNet [3]	✓	×	28.03	3.50	0.14	0.02	2172.66	20.23
DPRNN [23]	✓	×	168.77	2.62	4.68	0.46	7484.10	14.61
A-FRCNN-2 [17]	✓	×	10.31	5.14	0.10	0.03	847.24	21.55
A-FRCNN-4 [17]	✓	×	18.96	5.14	0.18	0.04	1535.25	21.55
A-FRCNN-8 [17]	✓	×	36.27	5.14	0.38	0.06	2909.77	21.55
AVConvTasNet [24]	✓	✓	43.57 *	16.45	0.43	0.07	2474.98 *	117.05
LAVSE [25]	✓	✓	206.51	32.89	0.55	0.05	2386.82	178.61
L2L [26]	✓	✓	93.58 *	17.40	0.57	0.04	834.68 *	116.83
VisualVoice [27]	✓	✓	19.70 *	77.75	1.98	0.20	5346.52 *	313.74
AVLIT-2 (Ours)	✓	✓	10.37	5.75	0.11	0.04	881.92	23.97
AVLIT-4 (Ours)	✓	✓	19.03	5.75	0.20	0.05	1570.98	23.97
AVLIT-8 (Ours)	✓	✓	36.35	5.75	0.48	0.08	2949.11	23.97

AVLIT-8 以仅 **5.75M 参数、23.97MB 推理内存**，在 NTCD-TIMIT 和 LRS3+WHAM! 上全面超越所有音频-only 及音视频基线，SI-SDRi 分别达 11.00 dB 和 12.42 dB，推理速度亦优于**参数量是其13倍的 VisualVoice**。



Dolphin: 离散唇语义 + 多尺度全局局部注意

视频编码器的“路径依赖”困境：AVSS 方法严重依赖大规模预训练视觉骨干网络（如 3D ResNet-18，预训练于唇读任务），带来了巨大的计算开销。

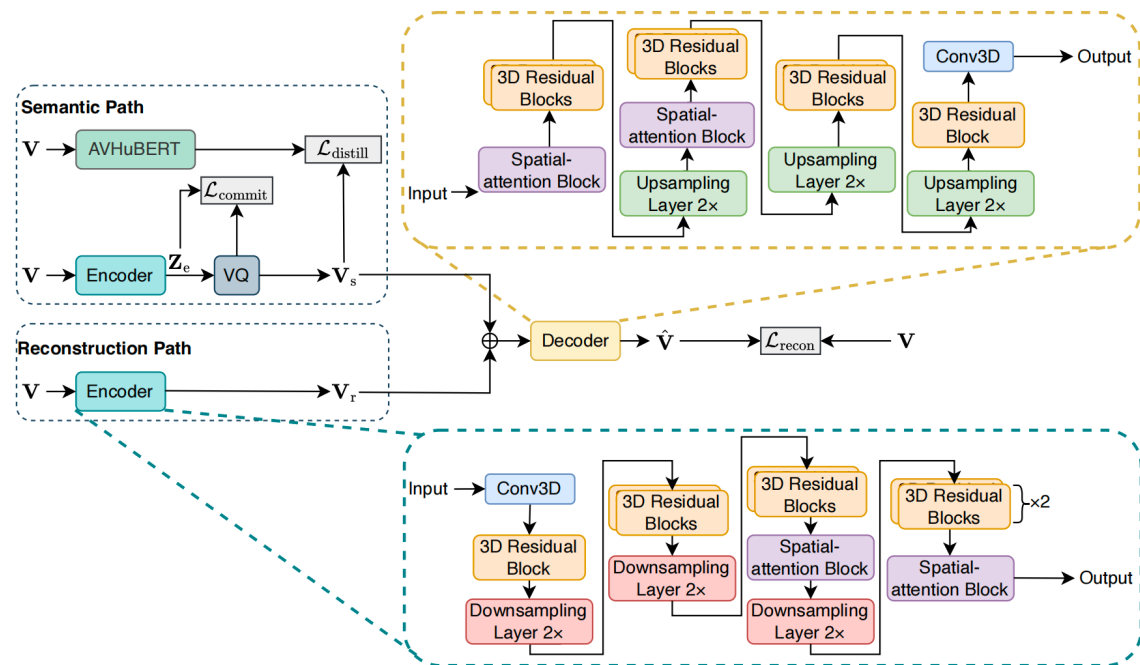
Methods	SI-SNRi $\uparrow$	SDRi $\uparrow$	PESQ $\uparrow$	Params (MB) $\downarrow$	MACs (G/s) $\downarrow$	Latency (ms) $\downarrow$	
						CPU	GPU
3D ResNet-18	17.0	17.1	3.30	11.19	7.95	2908.26	33.10
AE	15.2	15.4	3.15	0.05	0.17	2082.04	22.92
LipCoder	16.3	16.4	3.24	0.65	5.33	2473.84	25.53
DP-LipCoder	<u>16.8</u>	<u>16.9</u>	<u>3.29</u>	0.78	<u>2.38</u>	<u>2117.96</u>	<u>23.24</u>

DP-LipCoder 以不到 3D ResNet-18
7% 的参数和 30% 的计算量，仅牺牲 0.2 dB SI-SNRi

核心矛盾：轻量化 $\neq$ 能保留足够的语义对齐能力。这似乎形成了一个“铁律”——性能和效率不可兼得。

Dolphin 的思路：设计 DP-LipCoder，用向量量化 (VQ) + AV-HuBERT 知识蒸馏，将连续视频流映射为离散语义 token。这样做的直觉是：

- 离散化天然形成紧凑的“视觉词汇表”，迫使模型学到更有判别力的表示
- 蒸馏保证了与音频的语义对齐
- 双路径设计让重建路径保留说话人身份/表情等辅助线索，语义路径专注音频对齐





Dolphin: 离散唇语义 + 多尺度全局局部注意

音频分离器的迭代开销问题：现有高效方法（如 RTFSNet、AVLiT）虽然用了轻量级迭代分离器来减少参数，但迭代过程本身引入了显著的推理延迟

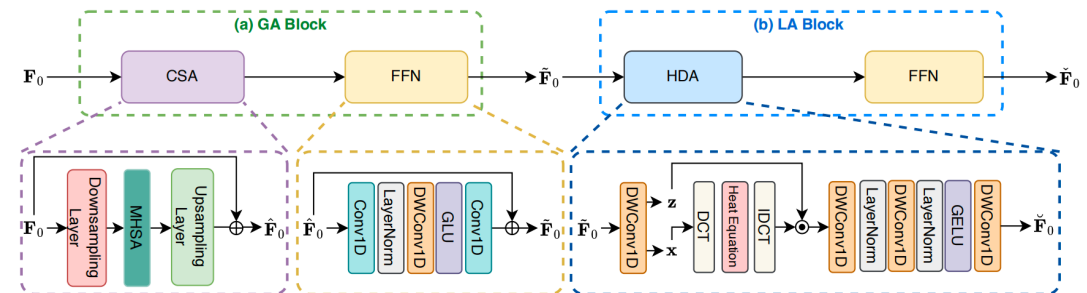
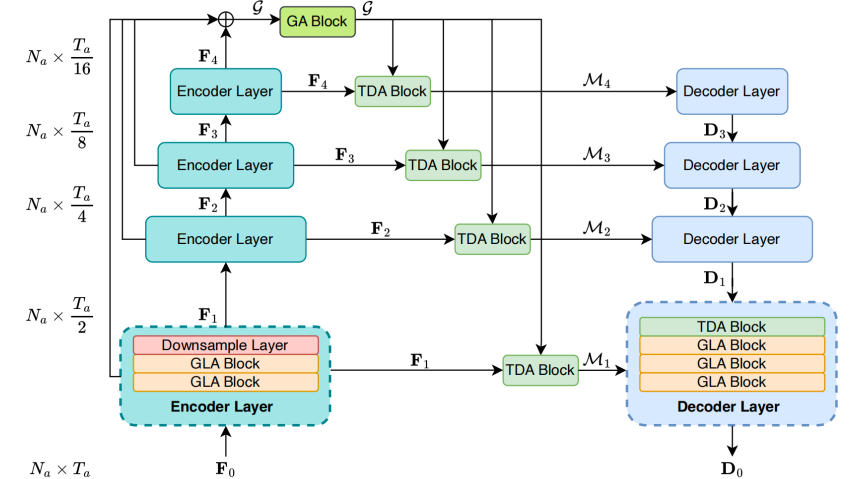
Methods	SI-SNR $\uparrow$	SDR $\uparrow$	PESQ $\uparrow$	Params (MB) $\downarrow$	MACs (G/s) $\downarrow$	Latency (ms) $\downarrow$	
						CPU	GPU
AV-TDANet-1	6.4	7.6	2.32	3.89	2.99	676.21	15.80
AV-TDANet-8	12.4	12.8	2.87	3.89	7.04	1185.80	44.67
AV-TDANet-16	12.8	13.2	2.89	3.89	11.68	2981.78	78.30
Dolphin	16.8	16.9	3.29	7.00	10.89	2117.96	33.24

迭代提升性能但牺牲速度；非迭代模型要么太重，要么性能不够。

Dolphin 的思路：只保留单次前向传播，但在每一层引入 GLA（Global-Local Attention）block 来弥补性能损失：

- GA block:** 用粗粒度自注意力（先降采样再做 MHSA）捕获全局长程依赖，复杂度降为原始的 $1/2^{2Q}$
- LA block:** 用热扩散方程推导的 HDA 注意力，在 DCT 频域做自适应多尺度滤波，建模局部结构，且只需学习少量缩放参数，避免大卷积核的参数开销和过拟合风险

TDANet 用 8 次迭代：MACs 从 2.99G 涨到 7.04G ($2.35\times$)
16 次迭代：MACs 涨到 11.68G ($3.9\times$)





Dolphin: 离散唇语义 + 多尺度全局局部注意

Methods	LRS2			LRS3			VoxCeleb2		
	SI-SNRi ↑	SDRi ↑	PESQ ↑	SI-SNRi ↑	SDRi ↑	PESQ ↑	SI-SNRi ↑	SDRi ↑	PESQ ↑
AV-ConvTasNet (Wu et al., 2019)	12.5	12.8	2.69	11.2	11.7	2.58	9.2	9.8	2.17
Visualvoice (Gao & Grauman, 2021)	11.5	11.8	2.78	9.9	10.3	2.13	9.3	10.2	2.45
AVLiT-8 (Martel et al., 2023)	12.8	13.1	2.56	13.5	13.6	2.78	9.4	9.9	2.23
CTCNet (Li et al., 2024a)	14.3	14.6	3.08	17.4	17.5	3.24	11.9	13.1	3.00
RTFS-Net (Pegg et al., 2023)	14.9	15.1	3.07	17.5	17.6	3.25	12.4	13.6	3.00
IINet (Li et al., 2024b)	16.0	16.2	3.23	18.3	18.5	3.28	13.6	14.3	3.12
AV-Mossformer2 (Zhao et al., 2025)	15.1	15.5	3.16	17.7	18.1	3.28	14.0	14.6	3.13
Swift-Net (Sang et al., 2025)	13.9	14.2	3.07	15.8	16.4	3.11	12.8	13.5	2.99
Dolphin (ours)	16.8	16.9	3.29	18.8	18.9	3.36	14.6	15.1	3.17

Dolphin 在 LRS2/LRS3/VoxCeleb2 三个基准上全面刷新 SOTA。

Methods	Parameters (M)↓		MACs (G)↓		Latency (ms)↓		Memory (MB)↓	
	w/o	w/	w/o	w/	CPU	GPU	RAM	GPU Inf.
AV-ConvTasNet (Wu et al., 2019)	13.70	25.01	6.94	14.90	342.15	10.54	95.47	129.51
Visualvoice (Gao & Grauman, 2021)	68.61	77.75	16.82	18.20	280.29	7.19	296.84	338.96
AVLiT-8 (Martel et al., 2023)	5.54	5.57	17.52	17.53	547.77	23.31	21.23	52.09
CTCNet (Li et al., 2024a)	6.87	18.06	75.80	83.75	1433.68	37.52	68.95	138.42
RTFS-Net (Pegg et al., 2023)	0.85	12.03	33.56	41.51	4021.93	56.59	48.40	249.49
IINet (Li et al., 2024b)	3.82	15.01	18.56	26.51	3213.82	142.30	57.31	148.14
AV-Mossformer2 (Zhao et al., 2025)	55.87	68.52	111.79	124.46	3083.86	62.30	262.45	398.76
Swift-Net (Sang et al., 2025)	1.47	12.66	18.24	26.18	2589.76	39.45	48.33	244.53
Dolphin (ours)	6.22	7.00	8.51	10.89	2117.96	33.24	26.84	251.12

Dolphin 以 7.0M 参数、10.89G MACs 实现了所有方法中最低的端到端计算开销



泽连斯基



特朗普

多模态融合部分小结

✓ 从“能不能融合”，走向“如何更轻、更准、更分层地融合”。

多模态融合的演进，本质上聚焦三个方向：**轻量视觉**、**选择性融合**、**多尺度交互**。

视觉端要“轻”，但不能丢语义

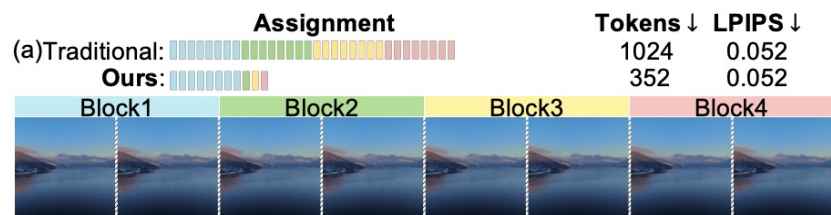
大模型视频 backbone 虽强，但部署代价高；因此 AVSS 正在从“重视觉 backbone”转向**轻量但语义对齐**的视觉表示。

融合要“选”，而不是“堆”

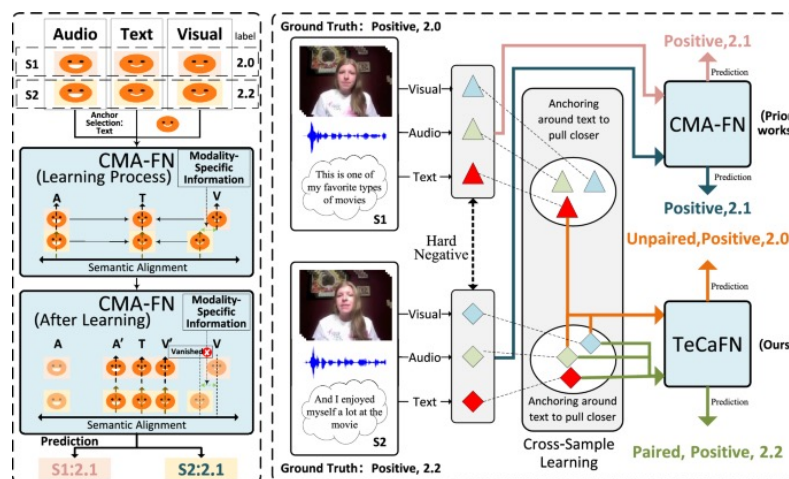
更高效的多模态融合方式，是用**门控/注意力**让视觉只增强和**目标语音相关的部分**，而不是做昂贵的全连接式跨模态交互。

交互要分层，而不是只在单尺度

单一时间尺度的融合不足以覆盖复杂语义关系；更优设计是在**粗细不同尺度上**进行交互，兼顾长程依赖与局部细节。



EVATok: Adaptive Length Video Tokenization for Efficient Visual Autoregressive Generation (CVPR 2026)



Text-centered cross-sample fusion network for multimodal sentiment analysis



因果部署

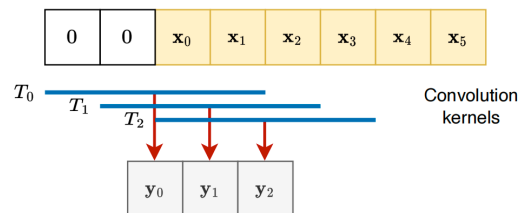
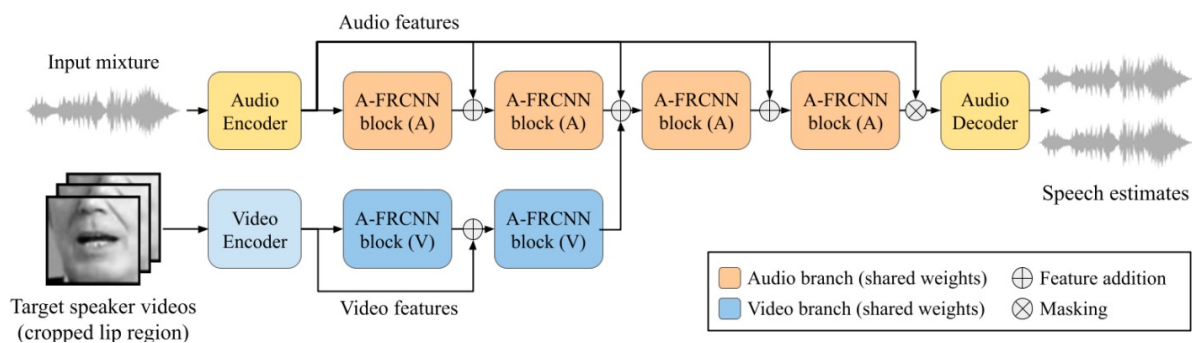
✓ 高效分离模型已经取得了明显进展；但新的问题是：这些高效结构能否真正工作在实时流式系统里？

离线高性能，不等于可实时部署

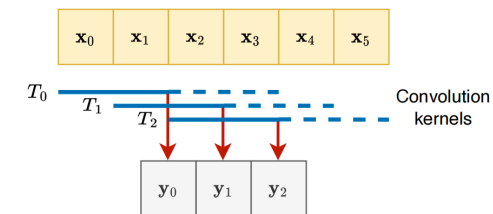
很多表现强的 结构依赖全局信息、复杂 backbone 或多轮迭代

“直接因果化”通常不够。

把非因果模型简单改成因版本，并不能自动得到真正轻量且实时可用的 AVSS 系统。



(b) Causal convolutional layer using padding method



(c) Causal convolutional layer using mask method

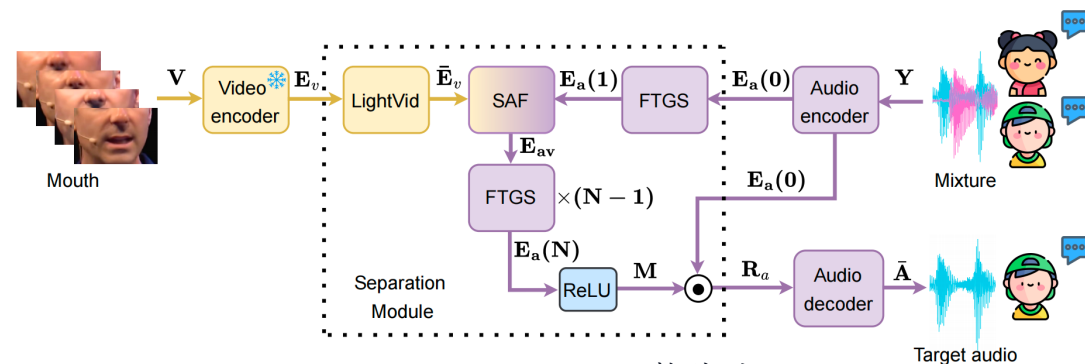
在因果约束下，模型既要只利用当前和历史信息，又要保留跨模态融合能力，同时高效建模历史时序依赖



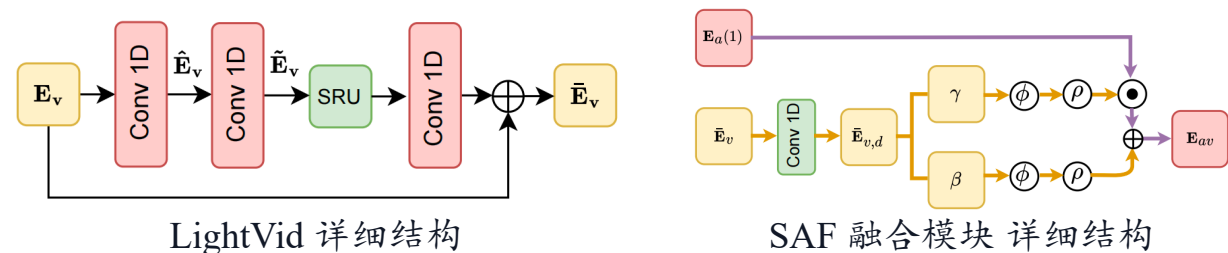
Swift-Net: 轻量因果 AVSS 的代表

- ✓ Swift-Net 面向 streaming AVSS 场景，围绕轻量视觉提取、高效音视频融合和历史信息建模三个环节，给出了一套因果方案。

1. 轻量视觉特征提取：提出 LightVid block，用 Conv + SRU 提取唇动时序特征，以更低计算量建模历史视觉信息。
2. 高效融合：提出 SAF block，通过选择性注意机制把视觉 cues 注入音频特征，只强化与目标说话人相关的部分。
3. 历史建模：提出 FTGS block，用 Grouped SRUs 在不同特征子空间中并行整合历史信息，兼顾长期依赖建模与低延迟推理。

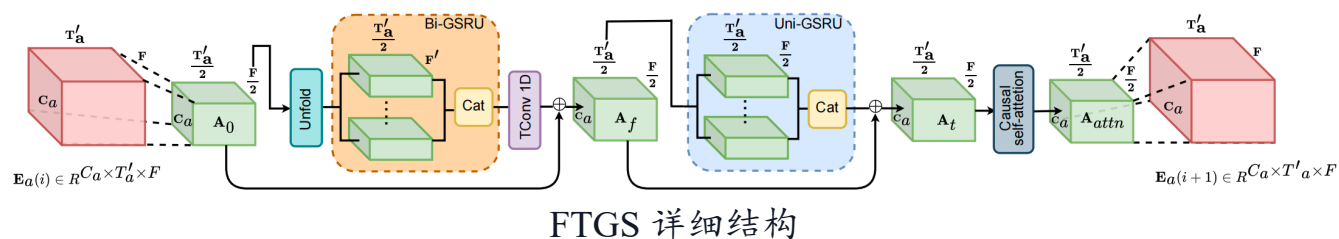


Swift-Net整体流程



LightVid 详细结构

SAF 融合模块 详细结构



FTGS 详细结构



Swift-Net: 轻量因果 AVSS 的代表

Model	LRS2-2Mix		LRS3-2Mix		VoxCeleb2-2Mix		Params (M)	MACs (G)	Time (ms)
	SI-SNRi	SDRi	SI-SNRi	SDRi	SI-SNRi	SDRi			
AV-ConvTasNet	9.8	10.3	12.5	12.8	6.9	8.2	11.0	15.0	16.1
AV-Sepformer	10.6	11.1	13.4	13.7	6.9	8.3	39.7	226.5	60.0
CTCNet	8.0	8.8	9.4	10.2	2.5	4.7	6.8	150.1	162.8
AV-DPRNN	10.2	10.7	11.9	12.3	7.8	9.0	1.3	4.5	5.8
AV-TF-GridNet	12.7	13.1	14.3	14.8	5.8	7.7	6.2	213.8	567.8
RTFSNet	12.4	12.8	14.0	14.4	8.4	10.7	0.6	49.8	178.8
Swift-Net-6	13.4	13.6	14.8	15.1	10.8	11.9	0.5	20.6	89.7
Swift-Net-9	13.6	13.8	15.4	15.8	11.4	12.4	0.5	28.6	130.7
Swift-Net-12	13.9	14.1	16.0	16.3	11.8	12.6	0.5	36.6	172.1

Swift-Net 在 LRS2、LRS3、VoxCeleb2 上都取得了最优或接近最优的 separation performance

Fusion Strategy	SI-SNRi	SDRi	Params (M)	MACs (G)
Concatenation	12.85	13.15	0.72	27.09
Addition	12.73	12.97	0.64	20.69
Cross-attention	13.10	13.40	26.38	27.24
SAF (ours)	13.34	13.64	0.53	20.68

在更少的计算量上，实现了更好的性能

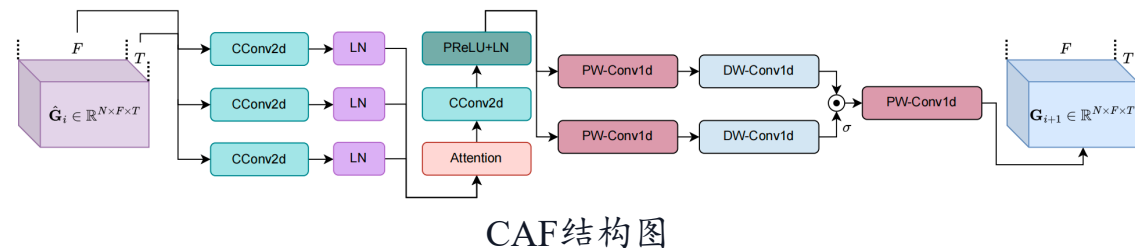
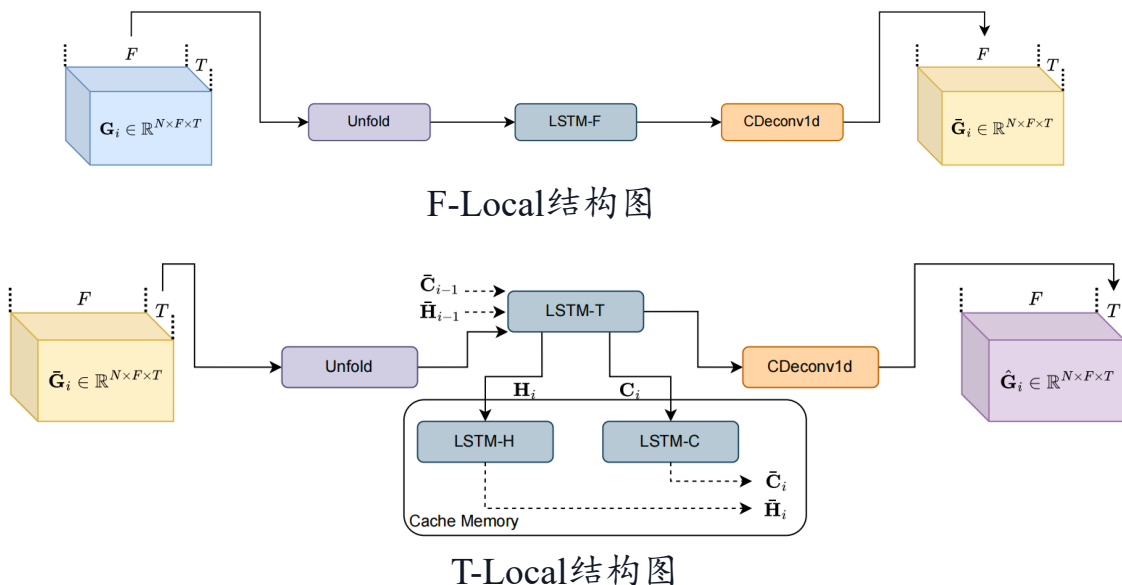
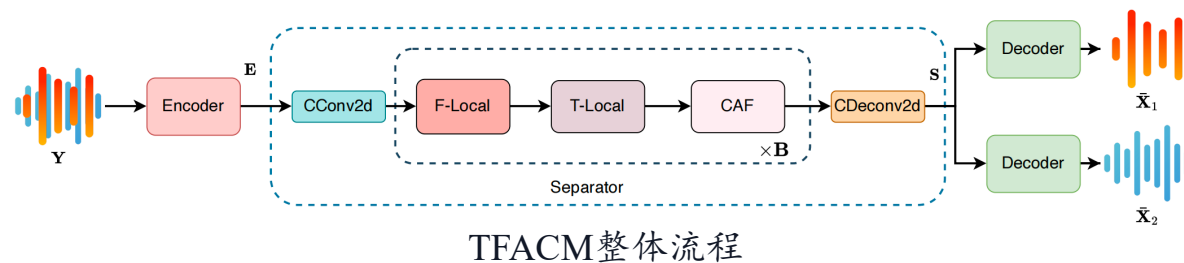


TFACM: cache memory 对实时分离意味着什么

✓ TFACM 的核心价值，不是“再做一个 causal 模型”，而是回答了实时分离里最关键的问题——在不能看未来的条件下，模型怎么把“过去的信息”真正存住、用好、再细化。

• **TFACM**: 能不能在时频域框架里，用更轻的方式把“过去”缓存下来，并持续服务后续时刻的推理。

- 整体思路可以概括为：**LSTM 建模频率** + **相对位置** + **cache memory 保留历史** + **CAR 强化因果时间建模**。





TFACM: cache memory 对实时分离意味着什么

Methods	WHAM!		WHAMR!		LibriMix		Params (M)	MACs (G/s)	Time (ms)
	SDRi	SI-SNRi	SDRi	SI-SNRi	SDRi	SI-SNRi			
DPRNN [2]	11.0	10.6	7.8	7.2	9.6	9.0	1.3	11.2	33.42
SkiM [13]	10.8	10.4	7.7	7.0	9.9	9.2	3.6	<u>11.1</u>	<u>39.01</u>
ReSepFormer [14]	11.7	11.3	7.6	6.9	4.1	3.3	8.0	6.3	47.46
TF-GridNet-Causal [7]	13.8	13.5	9.9	9.2	<u>11.5</u>	<u>10.8</u>	11.4	179.2	84.59
TFACM (Small)	12.0	10.9	<u>9.6</u>	<u>9.0</u>	10.2	9.3	0.5	19.4	45.17
TFACM (Large)	<u>13.3</u>	<u>13.0</u>	9.6	8.9	11.7	11.0	<u>1.0</u>	36.5	57.00

TFACM-Large 在 WHAM!/WHAMR!/LibriMix 上达到与 TF-GridNet-Causal 接近甚至局部更优的性能。

TFACM 用大约 8.8% 的参数、20.4% 的计算量，换来了接近 SOTA 的因果分离效果。

CM module	CAR module	SDRi	SI-SNRi
✓	✓	13.3	13.0
×	✓	11.8	11.4
✓	×	11.8	11.5
×	×	11.2	10.7

CM 与 CAR 两个模块缺一不可：只有同时保留历史缓存与时间因果细化时，模型才能达到最佳分离性能。



Swift-Net 与 TFACM 的共同点和区别

两者都在解决因果语音分离中的“历史信息利用不足”问题，但 Swift-Net 面向音视频分离，强调轻量化 AVSS 与因果化模板；TFACM 面向纯音频分离，强调时频域 attention + cache memory 的历史建模。

维度	Swift-Net	TFACM
是否使用音视频	是，AVSS	否，纯音频分离
是否强调因果模板	是，明确提出 causal transformation template	部分强调，更偏具体架构设计
历史机制	Grouped SRU 聚合历史信息	Attention + Cache Memory 显式缓存历史
核心优势	轻量化跨模态融合、适合实时 AVSS	时频域历史记忆强、复杂度低于 TF-GridNet-Causal
主要场景	看得见说话人的实时分离	纯语音实时分离

如果说 Swift-Net 解决的是“如何把 AVSS 做成真正可实时部署”，那么 TFACM 更侧重“如何在时频域里把历史记忆保存得更好”。



RNN-based online neural speech separation

如何把离线 Bi-RNN 分离模型改造成在线系统，并通过参数共享与训练策略，尽量缩小 online/offline 性能差距。

- ✓ 离线语音分离通常依赖双向 RNN，可同时利用过去和未来信息。
- ✓ 但在线系统不能访问未来信息，直接把 Bi-RNN 改成 Uni-RNN，性能往往明显下降。

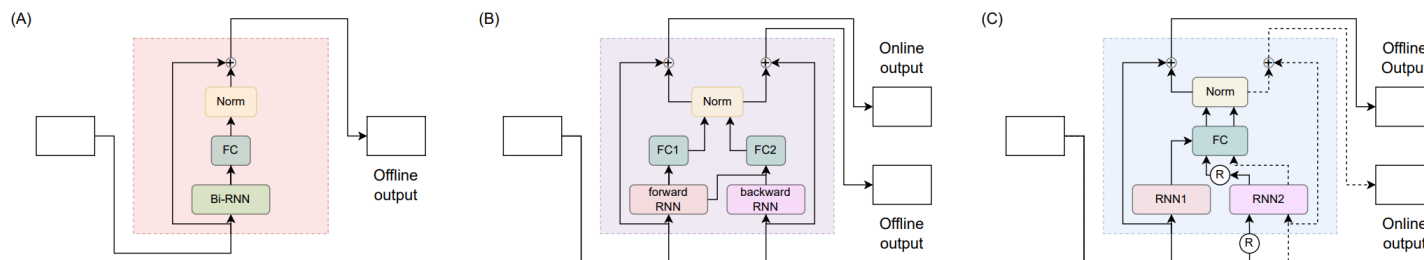
一个 Bi-RNN 层拆成 online path 和 offline path，使同一套参数既能做在线处理，也能做离线处理。

提出两种方式：

- Decomposition: 在线路径只保留 forward RNN，忽略 backward RNN。
- Reorganization: 将原本 backward RNN 重组为另一条可复用的 RNN，使同一层在在线/离线两种模式下共享更多参数。

Init	MT	Mode	Online	SI-SDRi		SDRi	
				TD	FD	TD	FD
✓	×	D R	✓	8.6 8.8	7.3 8.0	9.6 9.7	8.6 9.2
×	✓	D	×	9.4	9.7	10.2	10.5
		R	✓	8.3	8.0	9.4	9.2
×	✓	D	×	9.4	9.1	10.2	10.0
		R	✓	8.3	8.3	9.4	9.4
✓	✓	D	×	9.8	9.4	10.6	10.2
		R	✓	8.5	7.4	9.4	8.6
✓	✓	D	×	9.8	9.5	10.6	10.4
		R	✓	8.5	8.0	9.5	9.1

结合离线初始化与多任务训练后，在线模型的性能下降被明显缓解，其中 reorganization (R) 方案整体优于 decomposition (D)，尤其在频域模型上更稳定。





因果语音分离模型设计的规律

1. 必须显式管理历史信息

- a) 在线系统不能访问未来信息，因此必须设计替代机制来保留过去。
- b) 常见做法包括 hidden state 传递、cache memory、grouped recurrent modeling、历史摘要向量等。

2. 缓存不是附属模块，而是核心能力

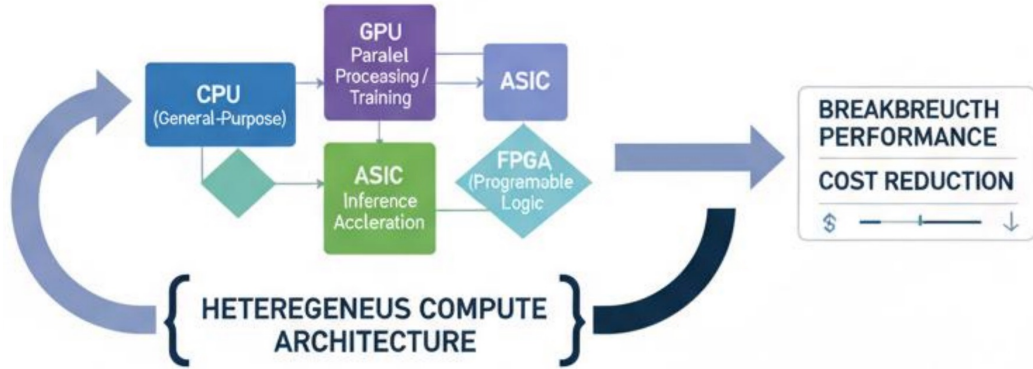
- a) TFACM 表明，仅靠因果 attention 或单向 RNN 还不够，必须考虑如何跨块、跨层传递历史状态。
- b) 历史缓存做得好，往往比单纯堆大模型更有效。

3. 训练方式必须与推理方式一致

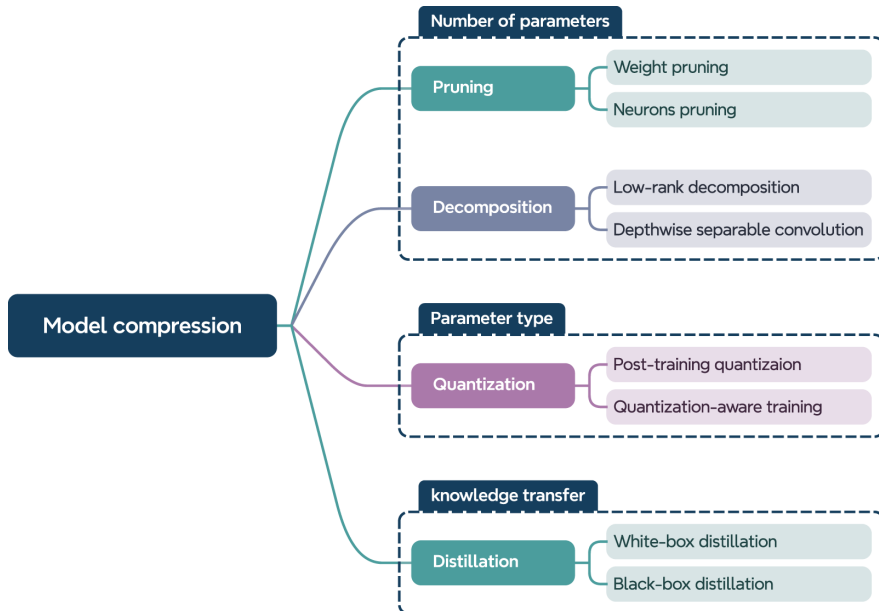
- a) 在线模型若直接从零训练，性能常明显落后于离线模型。
- b) 更有效的做法是：离线初始化、在线/离线多任务训练，或让模型在训练阶段就暴露于真实在线路径。



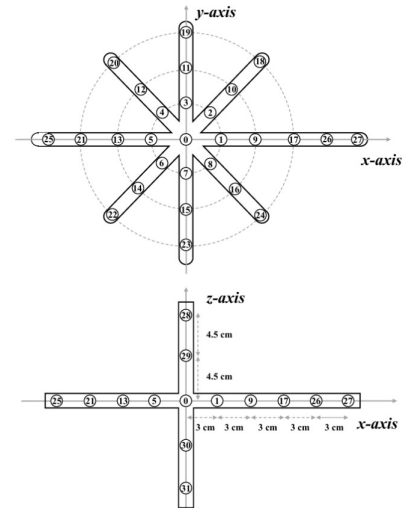
部署时还有什么问题没解决？



系统弹性是否足够。



已有模型如何压缩到可部署形态，仍是工程难点。

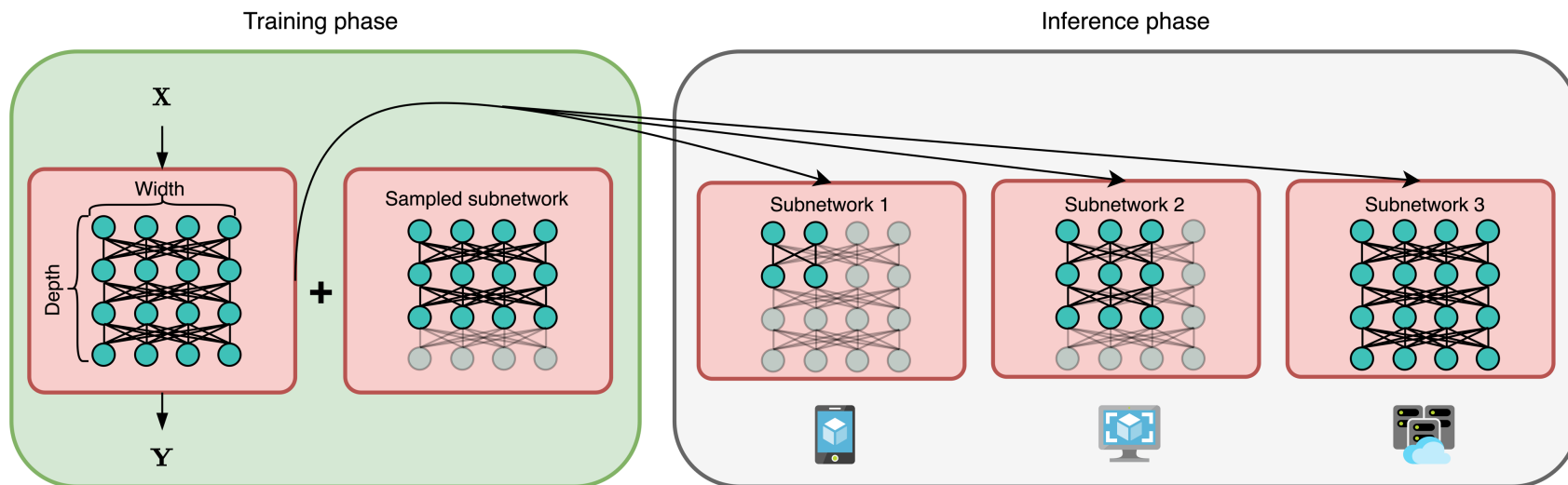


如果训练数据和评测环境过于理想化，模型上线后仍可能失效。



Subnetwork-to-Go: 面向多设备部署的弹性神经网络

现有问题：模型太大难部署，模型太小性能不足，而为不同设备分别训练又代价过高。



动态训练和不同规模推理

- 宽度可变：可根据设备算力选择通道规模
- 深度可变：可根据时延要求选择网络层数
- 统一训练：避免为每个配置重复训练模型

(w, d)	Separately trained	SNR (dB)	Training time (GPU hrs)
$w=1, d=1$	✓	4.92	57
$w=1, d=1$	×	5.01	-
$w=1, d=12$	✓	7.94	188
$w=1, d=12$	×	8.31	-
$w=16, d=12$	✓	8.36	350
$w=16, d=12$	×	8.60	-
Full network	-	-	437

实验表明，在不同宽度/深度设置下，子网络的 SNR 均高于独立训练模型，同时避免了重复训练开销。

用一个可弹性伸缩的模型，替代多个固定模型，实现高效训练与按需推理。

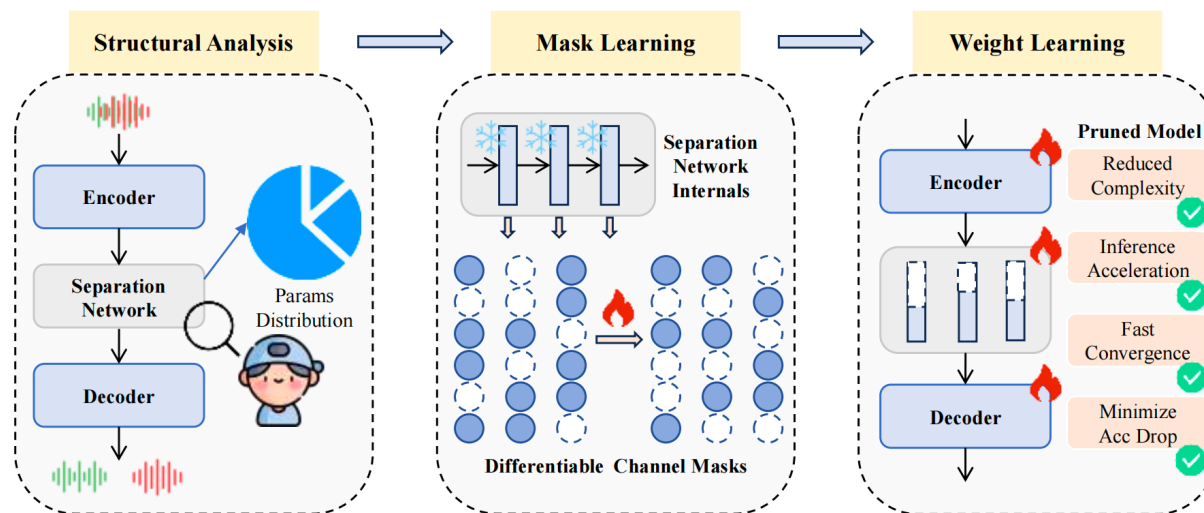


SepPrune: 结构化剪枝进入语音分离

现有问题：现有语音分离模型重效果、轻效率；实时通信与边缘部署需要更低延迟、更小计算开销

SepPrune 是首个面向语音分离模型的 structured pruning 框架

1. 先做结构计算分析，定位主要计算瓶颈
2. 再通过可微分掩码学习进行通道选择
3. 最后执行 channel pruning，并用少量 fine-tuning 恢复性能



Model	Total Params	Total FLOPs	Params of the SM	FLOPs of the SM	SM Parameter Ratio	SM FLOPs Ratio
A-FRCNN-12	5.13 M	28.58 GMac	4.22 M	26.56 GMac	82.31%	92.94%
A-FRCNN-16	5.13 M	37.44 GMac	4.22 M	35.42 GMac	82.31%	94.59%
SuDoRM-RF1.0x	2.72 M	4.65 GMac	2.35 M	3.57 GMac	86.50%	76.86%
TDANet	2.35 M	4.77 GMac	2.29 M	4.59 GMac	97.44%	96.22%
TDANet Large	2.35 M	9.20 GMac	2.29 M	9.09 GMac	97.44%	98.84%

计算瓶颈主要集中在分离网络



SepPrune: 结构化剪枝进入语音分离

Model	Type	Train				Inference			
		GPU Time	GPU Memory	Speed Up	Memory Savings	GPU Time	GPU Memory	Speed Up	Memory Savings
A-FRCNN-12	Original Model	160.67ms	3714MB			104.23ms	642MB		
	Pruned Model	155.14ms	2284MB	1.04×	38.50%	95.38ms	628MB	1.09×	2.20%
TDANet	Original Model	435.75ms	5130MB			223.59ms	640MB		
	Pruned Model	409.56ms	3994MB	1.06×	22.14%	207.52ms	639MB	1.08×	0.01%
SuDoRM-RF1.0x	Original Model	157.43ms	4084MB			89.87ms	612MB		
	Pruned Model	150.97ms	2034MB	1.04×	50.20%	79.25ms	602MB	1.13×	1.60%

剪枝后的模型在训练与推理阶段都带来实际效率收益：训练最高**节省 50.2% GPU 显存**，推理速度最高**提升 1.13×**，验证了 SepPrune 不仅压缩有效，而且具备真实部署价值。

Model	Method	LRS2-2Mix				Libri2Mix				EchoSet			
		Params	FLOPs	SDRi	SI-SDRi	Params	FLOPs	SDRi	SI-SDRi	Params	FLOPs	SDRi	SI-SDRi
A-FRCNN-12	-	5.13	28.58	10.90	10.50	5.13	28.58	15.54	15.03	5.13	28.58	8.56	7.66
	Random	3.06	16.52	12.03	11.68	3.09	16.76	15.03	14.78	3.08	16.57	7.41	6.26
	Hrank	3.06	16.52	12.45	12.11	3.09	16.76	<u>15.32</u>	<u>15.00</u>	3.08	16.57	<u>8.01</u>	<u>6.99</u>
	UDSP	3.06	16.52	12.49	12.08	3.09	16.76	15.23	14.94	3.08	16.57	7.93	6.94
	SepPrune	3.06	16.52	12.59	12.25	3.09	16.76	15.60	15.10	3.08	16.57	8.12	7.13
TDANet	-	2.35	4.77	12.74	12.45	2.35	4.77	13.42	12.92	2.35	4.77	8.78	7.93
	Random	1.92	4.33	11.91	11.52	1.67	4.07	13.02	12.08	1.64	4.04	7.51	6.49
	Hrank	1.92	4.33	12.33	12.01	1.67	4.07	13.23	12.46	1.64	4.04	7.69	6.67
	UDSP	1.92	4.33	12.46	12.26	1.67	4.07	<u>13.56</u>	<u>12.79</u>	1.64	4.04	<u>7.99</u>	<u>7.03</u>
	SepPrune	1.92	4.33	12.72	12.41	1.67	4.07	13.70	13.21	1.64	4.04	8.52	7.62
SuDoRM-RF1.0x	-	2.72	4.65	11.43	11.10	2.72	4.65	13.52	12.99	2.72	4.65	7.82	6.77
	Random	1.54	2.91	9.53	9.06	0.98	2.09	12.14	11.82	1.11	2.28	6.52	5.74
	Hrank	1.54	2.91	<u>10.29</u>	<u>9.89</u>	0.98	2.09	<u>12.36</u>	12.01	1.11	2.28	6.85	5.89
	UDSP	1.54	2.91	10.12	9.67	0.98	2.09	12.31	12.04	1.11	2.28	7.04	5.98
	SepPrune	1.54	2.91	10.37	9.98	0.98	2.09	12.79	12.24	1.11	2.28	7.29	6.07

SepPrune 在 LRS2-2Mix、Libri2Mix 和 EchoSet 上、针对 A-FRCNN-12、TDANet、SuDoRM-RF1.0x 三种骨干，均在相同剪枝规模下取得了**最佳或最优的 SDRi / SI-SDRi**。



SonicSim: 更真实的复杂混响评测

为什么评测环境很重要? ——SonicSim 的真实场景挑战

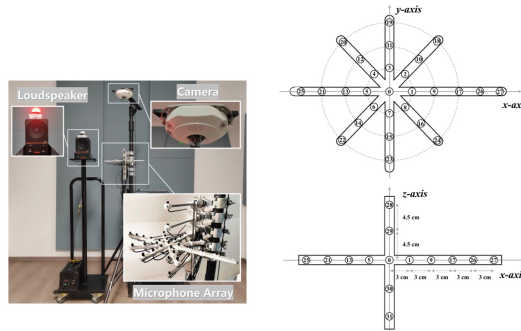


Table 4: Benchmark experiments of speech enhancement.

Baseline	Training Data		Static Speaker						Moving Speaker					
	speech	noise	WB-PESQ	SI-SDR	MOS-SIG	MOS-BAK	MOS-OVR	CER	WB-PESQ	SI-SDR	MOS-SIG	MOS-BAK	MOS-OVR	CER
unprocessed	-	-	1.15	-9.8	2.00	1.72	1.51	19.9	1.11	-9.1	1.79	1.54	1.36	23.8
FaSNet-TAC [10]	sim	sim	1.38	-3.4	2.67	3.19	2.22	27.1	1.33	-2.5	2.60	3.12	2.14	29.7
	sim	real	1.49	-1.7	2.83	3.23	2.35	22.4	1.42	-1.5	2.78	3.20	2.29	25.7
	real	sim	1.47	0.8	2.67	3.09	2.18	23.7	1.40	0.5	2.58	3.05	2.10	28.2
	real	real	1.51	1.3	2.80	3.34	2.35	22.4	1.43	1.1	2.73	3.28	2.27	26.3
SpatialNet [28]	sim	sim	1.40	-8.4	3.09	2.62	2.28	19.2	1.33	-7.9	3.06	2.53	2.23	23.2
	sim	real	1.45	-2.6	2.58	2.35	1.95	23.0	1.38	-2.6	2.54	2.25	1.89	26.5
	real	sim	1.96	3.8	3.09	3.06	2.45	17.3	1.80	3.0	3.00	2.99	2.36	21.2
	real	real	2.10	6.1	3.05	3.51	2.62	16.0	1.90	3.8	2.96	3.45	2.52	21.5

RealMAN: <https://arxiv.org/abs/2406.19959> (录制成本极高)

Datasets	Geometry	Occlusion	Material	Scalability	Cost	Tools	Src Type	Tasks
WHAMR! 2020	Cuboid	✗	✗	✓	Low	✓	Static	SS/SE
LibriCSS 2020b	Cuboid	✓	✓	✗	High	✗	Static	SS
DNS Challenge 2020	Cuboid	✗	✗	✓	Low	✓	Static	SE
Chime6 2020	Variable	✓	✓	✗	High	✗	Static	SS
LRS2 2023	Variable	✓	✓	✗	High	✗	Static	SS
RealMan 2024	Variable	✓	✓	✗	High	✗	Dynamic	SE
SonicSet (ours)	Variable	✓	✓	✓	Low	✓	Dynamic	SS/SE

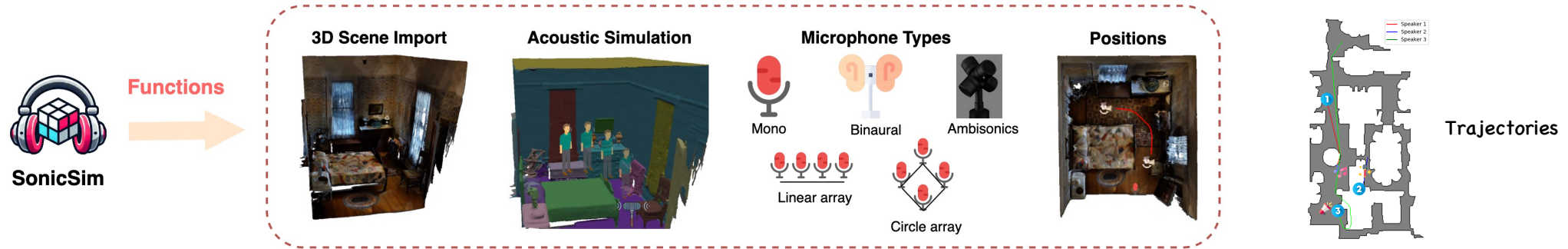
图标	挑战	传统数据集的问题
🧱 障碍物遮挡	声源与麦克风之间存在物体遮挡	图像源法无法处理任意形状障碍物
🏠 复杂房间几何	真实空间形状多样 (阶梯厅、走廊...)	大多假设长方体房间
🪵 材质吸声差异	不同材料有不同吸声/散射系数	忽略材质, 声学失真

+ 移动声源: 真实场景中说话人会走动, 而现有数据集几乎全是静态声源

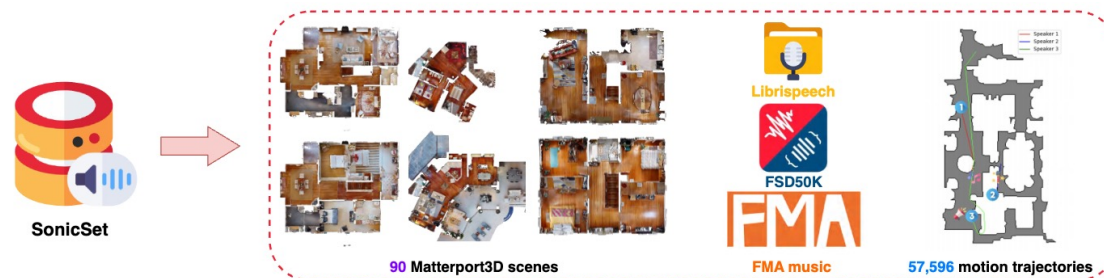
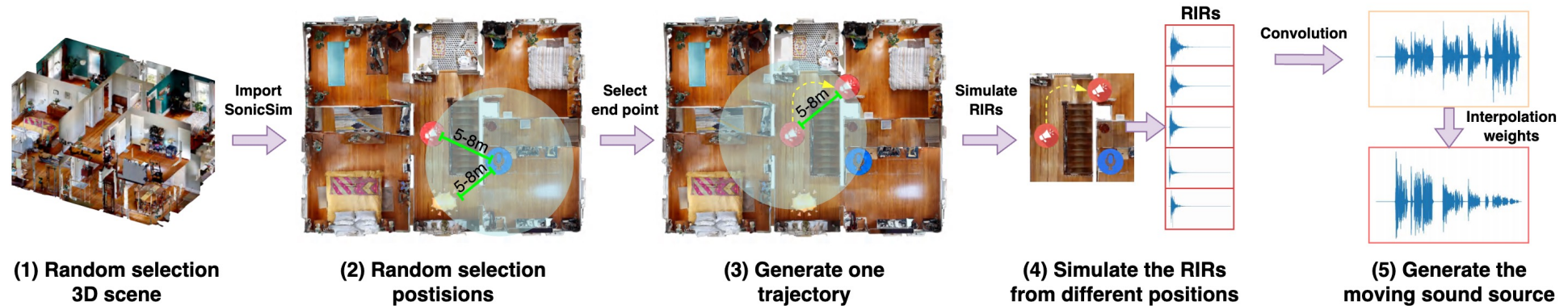


SonicSim: 更真实的复杂混响评测

SonicSim 平台示意图



构造方法



Datasets	Speakers	Utterances	Duration (h)	Noise	Reverb	Dynamic
Speech separation and enhancement						
WHAM! (2019)	191	28,000	43	✓	✗	✗
WHAMR! (2020)	191	28,000	43	✓	✓	✗
SonicSet (ours)	1001	57,596	960	✓	✓	✓



SonicSim: 更真实的复杂混响评测

语音分离Benchmark

Method	SI-SNR ↑	SDR ↑	NB-PESQ ↑	WB-PESQ ↑	STOI (%) ↑	WER (%) ↓
Conv-TasNet	2.18/2.45/3.02	3.09/3.24/4.82	1.98/2.03/2.12	1.27/1.31/1.55	59.73/60.33/65.32	98.33/87.04/74.85
DPRNN	2.23/2.81/3.71	2.91/3.54/4.34	1.92/2.05/2.18	1.25/1.32/1.62	60.02/64.76/70.13	91.05/72.63/55.34
DPTNet	4.76/5.53/7.42	5.63/6.68/8.52	2.12/2.32/2.68	1.87/1.91/2.12	71.83/73.42/77.73	51.19/48.18/38.17
SuDoRM-RF	3.44/4.79/5.85	4.22/5.26/6.78	2.08/2.18/2.41	1.58/1.62/1.87	67.77/72.38/73.39	65.22/55.47/48.54
A-FRCNN	3.65/4.87/6.02	4.38/5.67/6.80	2.08/2.21/2.43	1.65/1.68/1.90	69.10/70.23/73.85	68.27/54.32/47.93
SKIM	2.31/2.87/3.33	2.99/3.67/4.13	1.97/2.03/2.07	1.37/1.45/1.63	62.11/64.42/66.67	77.02/70.54/53.84
TDANet	3.90/5.15/6.11	4.71/5.98/7.10	2.15/2.28/2.50	1.72/1.69/1.94	69.95/71.14/74.59	58.40/54.39/43.60
BSRNN	3.68/5.09/6.15	4.46/5.96/6.93	2.10/2.22/2.59	1.79/1.71/2.07	71.26/73.22/76.06	57.63/53.59/48.64
TF-GridNet	6.63/8.27/11.82	7.52/9.10/12.59	2.54/2.71/3.05	2.09/2.28/2.40	79.21/80.34/85.50	34.64/30.21/20.50
Mossformer	5.72/7.94/10.72	6.54/8.78/11.63	2.51/2.60/2.97	2.18/2.23/2.31	75.38/79.32/81.21	47.33/33.84/30.44
Mossformer2	5.87/7.81/10.57	6.66/8.65/11.25	2.56/2.58/2.98	2.23/2.21/2.35	75.50/78.94/81.00	42.94/33.09/29.57

Table 2: Comparative performance evaluation of models trained on different datasets using real-recorded audio with *environmental noise*. The results are reported separately for “*trained on LRS2-2Mix*”, “*trained on Libri2Mix*” and “*trained on SonicSet*”, distinguished by a slash. The relative length is indicated below the value by horizontal bars.

语音增强Benchmark

Method	SDR ↑	WB-PESQ ↑	MOS Sig ↑	MOS Bak ↑	MOS Overall ↑	CER (%) ↓
DCCRN	-1.10/1.87/1.95	1.11/1.24/1.26	2.26/3.25/2.44	2.90/2.12/3.36	1.90/2.19/2.27	50.65/37.56/21.70
Fullband	-1.55/1.18/1.37	1.04/1.07/1.27	2.50/2.84/2.53	2.22/2.61/3.47	2.09/2.19/2.46	51.67/39.71/20.94
FullSubNet	-0.75/1.48/1.92	1.10/1.19/1.30	2.40/2.73/2.69	2.94/2.76/3.48	1.94/2.24/2.46	49.23/32.39/19.15
Fast-FullSubNet	-1.55/1.38/1.37	1.08/1.15/1.31	2.45/3.13/2.67	2.09/2.09/3.48	2.04/1.97/2.59	49.97/40.17/20.08
FullSubNet+	-0.58/1.64/1.32	1.11/1.27/1.28	2.44/2.51/2.59	2.09/2.87/3.52	2.07/2.31/2.46	45.22/23.98/20.48
TaylorSENet	1.06/1.78/2.26	1.21/1.33/1.31	2.44/2.68/2.47	2.09/2.63/2.43	2.10/2.23/2.33	42.55/28.19/20.64
GaGNet	-0.13/1.65/2.10	1.07/1.27/1.30	2.62/2.53/2.46	2.44/3.16/2.41	2.32/2.35/2.40	44.39/34.77/21.09
G2Net	0.01/1.52/2.10	1.10/1.21/1.29	2.76/2.75/2.45	2.21/2.53/2.41	2.07/2.17/2.41	55.12/42.98/21.67
Inter-SubNet	-1.62/1.35/1.61	1.09/1.29/1.34	2.13/2.67/2.70	3.83/2.88/3.47	1.83/2.40/2.51	47.73/22.96/18.73

Table 5: Comparative performance evaluation of models trained on different datasets using the Real-MAN dataset. The results are reported separately for “*trained on VoiceBank-DEMAND*”, “*trained on DNS Challenge*” and “*trained on SonicSet*”, distinguished by a slash.

在真实录音数据集上，SonicSet 训练的模型在**语音分离（WER ~20%）**和**语音增强（CER ~19%）**任务中均全面优于 Libri2Mix 和 DNS Challenge 等现有数据集，证明更逼真的声学仿真能有效缩小 sim-to-real gap。

“仿真越真实，迁移越可靠——SonicSim 让合成数据首次支持移动声源的真实场景评测”



Hive: 高纯度数据带来的数据效率

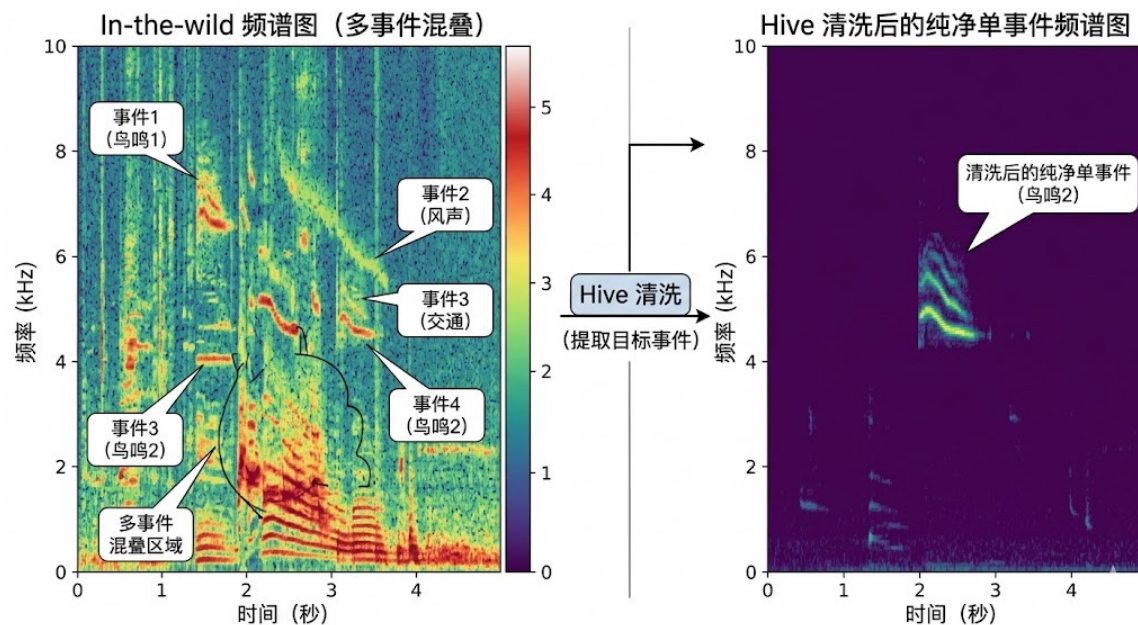
为什么现有数据会"误导"模型?

❌ 缺陷一：事件共现（Co-occurrence）

"Rain" 音频片段里 → 实际包含：雨声 + 风声 + 车声
模型学到：背景噪声 = 雨的特征
结果：分离时把风声也留下了

❌ 缺陷二：弱标签（Weak Label）

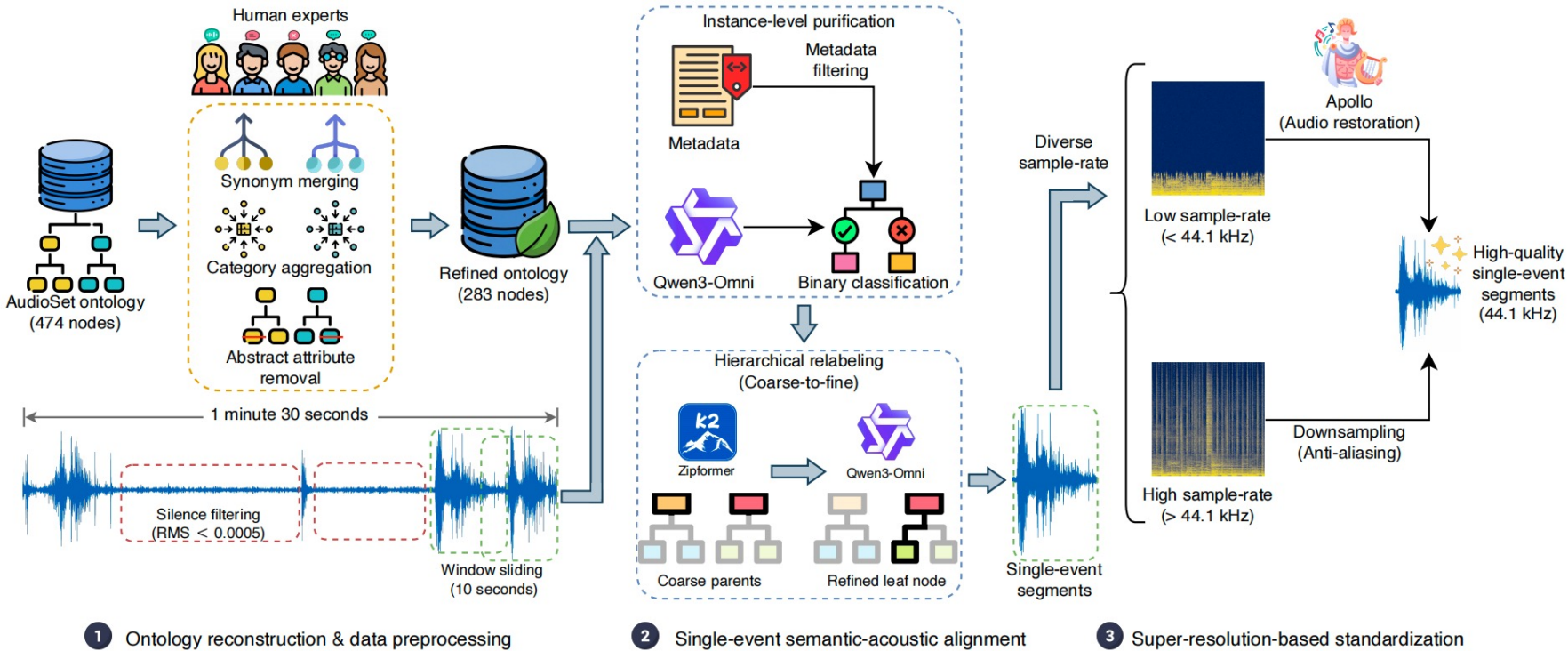
AudioSet 标注 "Rain" → 无法说明哪段是纯净雨声
模型建立：噪声-类别 伪相关
结果：泛化能力差



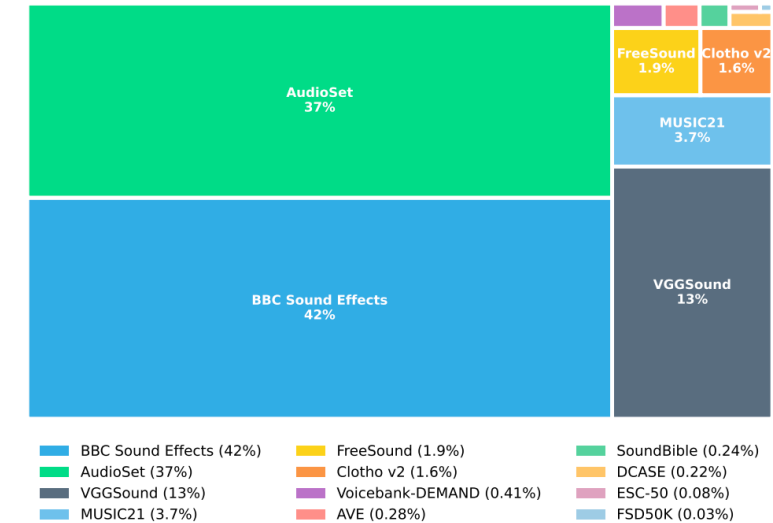


Hive: 高纯度数据带来的数据效率

Hive 三阶段 Pipeline



Hive 使用的源数据





Hive: 高纯度数据带来的数据效率

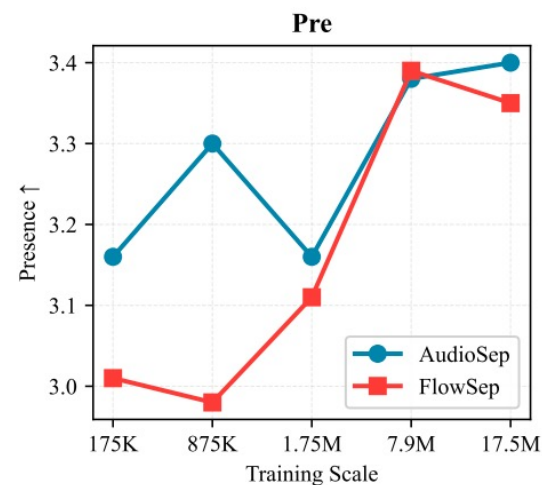
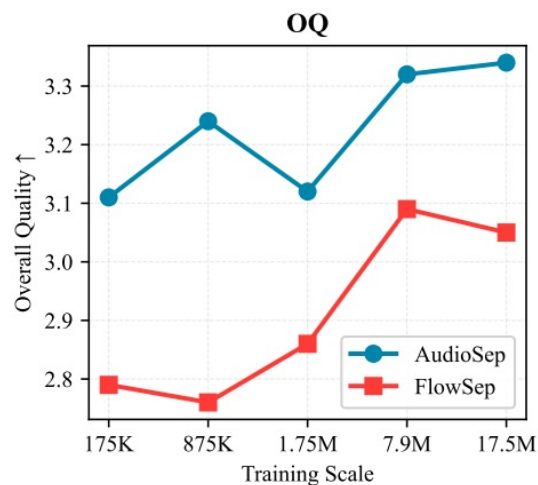
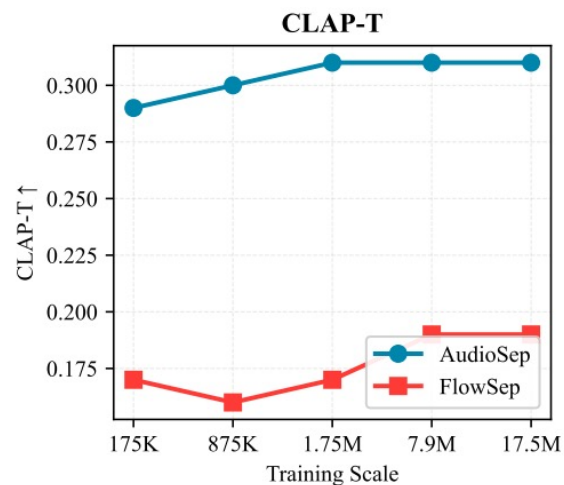
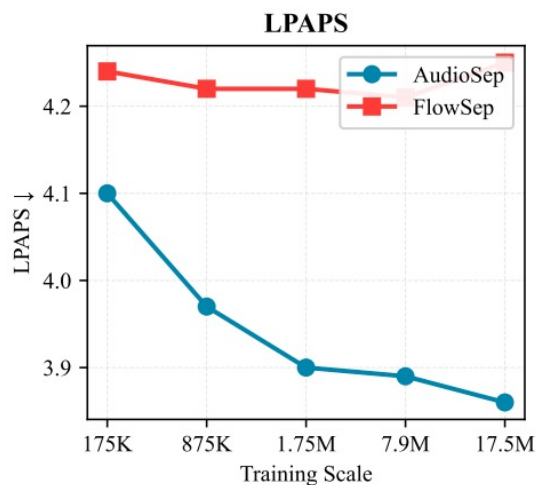
USS-Bench 测试集

训练数据	规模	SDR ↑	FAD ↓	CLAP-A ↑	OQ ↑
AudioSep (原始 in-the-wild)	14,100h	-1.86	0.91	0.58	2.97
SAM-Audio (原始)	~100万h	—	0.90	0.57	3.47
AudioSep (Hive)	2,400h	+2.29	0.75	0.69	3.56

MUSDB18-HQ 测试集

训练数据	规模	SDR ↑	FAD ↓	OQ ↑
AudioSep (原始)	14,100h	-1.01	0.97	2.39
SAM-Audio	~100万h	—	0.83	3.24
AudioSep (Hive)	2,400h	+1.36	0.91	3.01

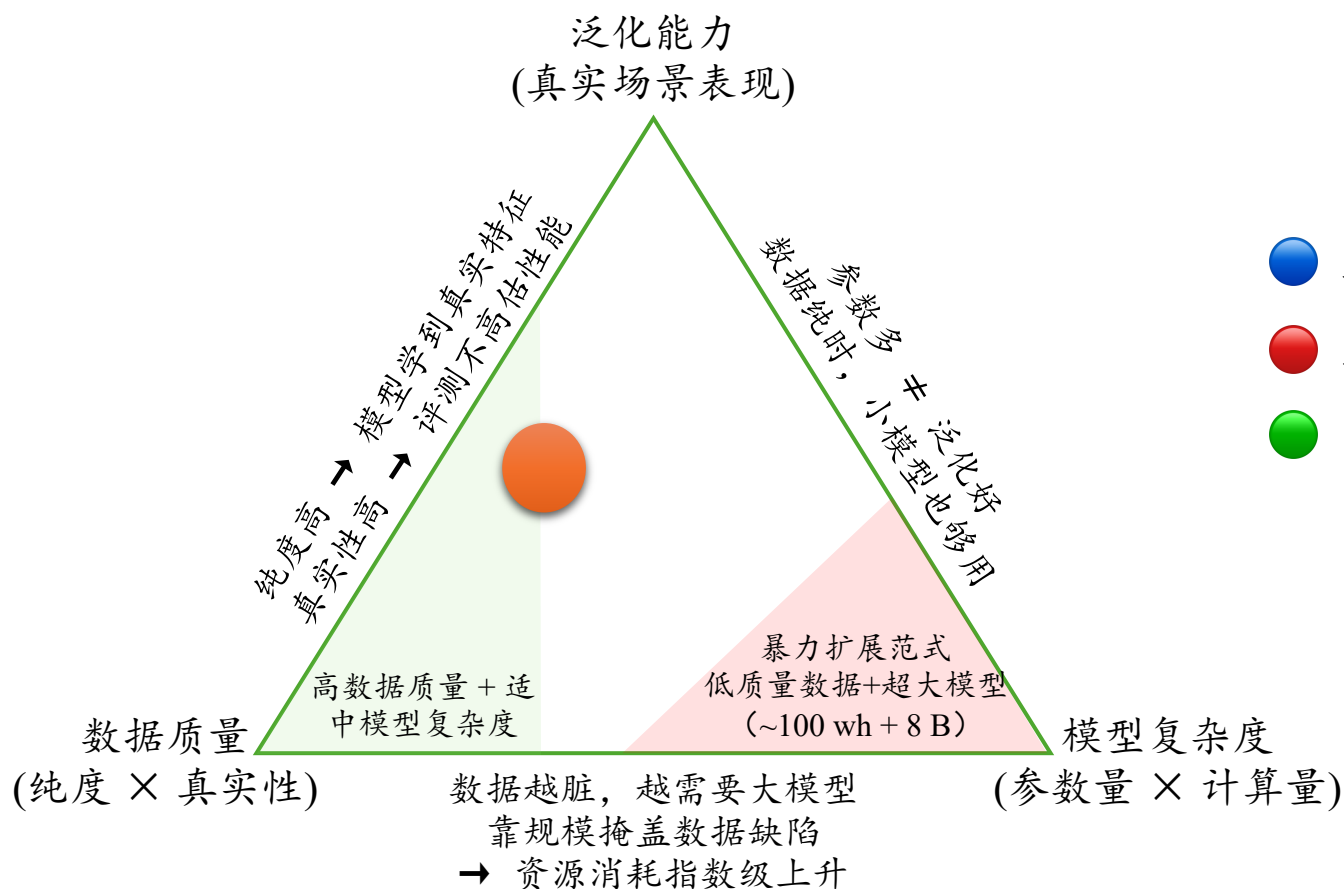
Hive 仅用 SAM-Audio 数据量的 **0.2%**，在 FAD (0.75 vs 0.90) 和 CLAP-Audio (0.69 vs 0.57) 上反超百万小时规模模型



875K 高纯度样本 (~1,000小时) 已超越 14,100 小时的噪声数据



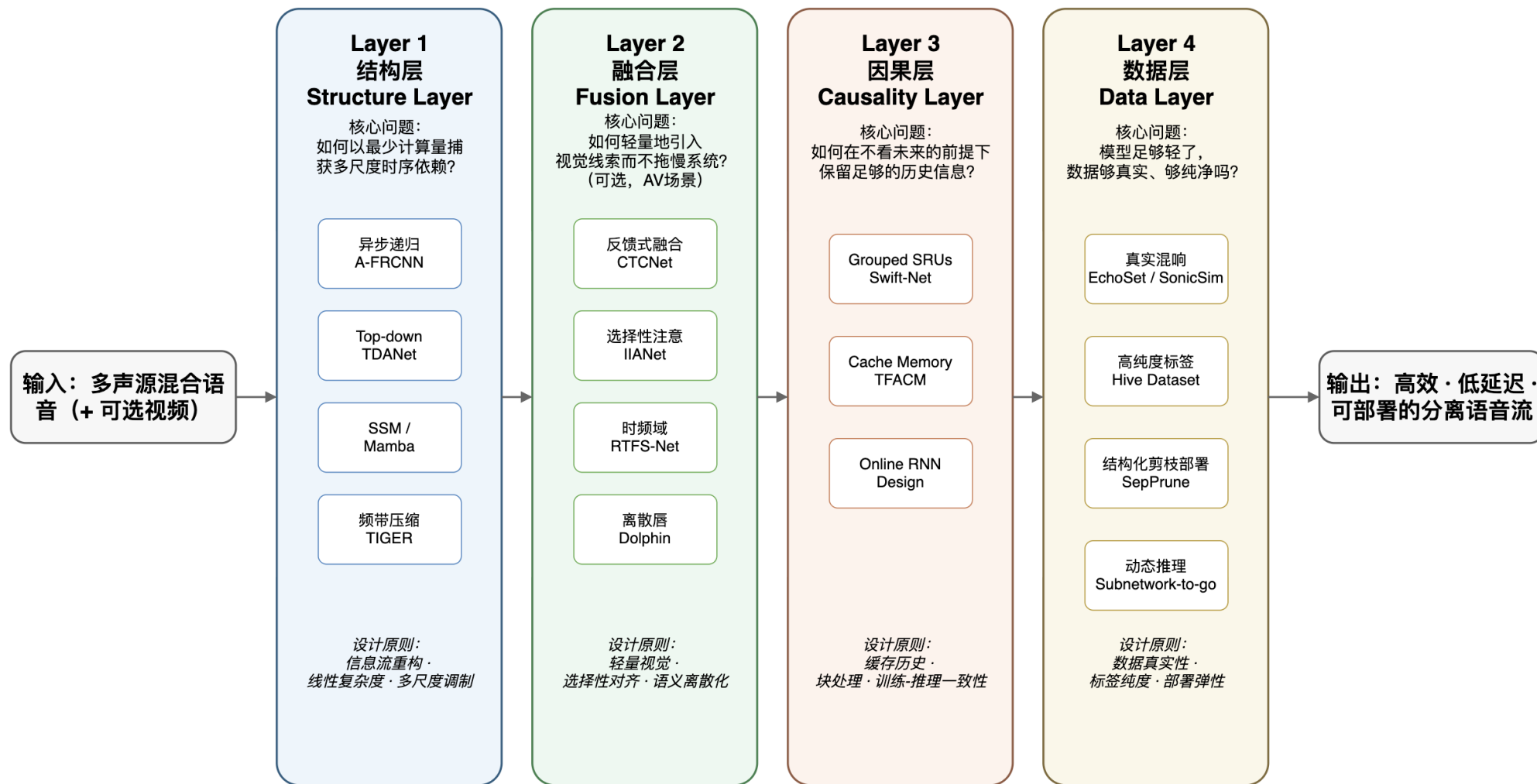
三角关系：数据质量如何重塑模型设计的边界



- 提升数据质量 = 最高性价比的模型改进方式
- 控制模型复杂度 = 数据纯净时, 无需暴力扩参
- 追求真实泛化 = 评测标准决定研究方向的正确性

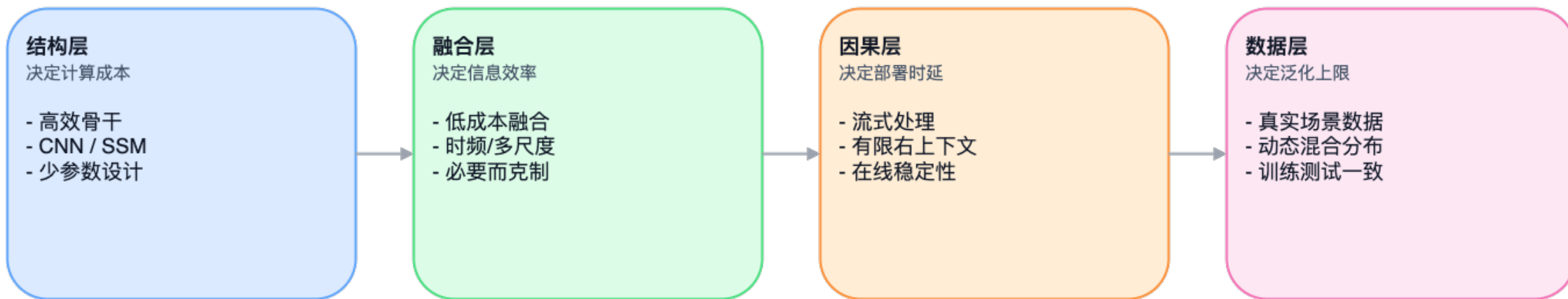


统一框架：高效语音分离的四层结构



- 1) 信息流设计是高效 backbone 的核心。
- 2) 视觉应该变成轻量语义提示。
- 3) 因果系统的关键是历史记忆。
- 4) 数据纯度与真实性直接决定部署价值。

未来展望



- 结构层：决定基础计算成本，需转向高效骨干与低复杂度建模
- 融合层：决定信息利用效率，关键在低成本的时频与多尺度融合
- 因果层：决定实时部署能力，需要流式处理与有限上下文建模
- 数据层：决定真实场景泛化，训练分布应尽量接近部署环境



感谢SJTU X-LANCE lab的邀请
感谢各位的聆听，欢迎大家讨论交流。

主页：cslikai.cn

邮箱：li-k24@mails.tsinghua.edu.cn